



Logiciel de gestion des bornes de recharge pour véhicules électriques

Par

DAMA Prisca Fehiarisoa

Mémoire présenté

en vue de l'obtention du grade de Licence

en Informatique

Option Développement

Novembre 2024

Jury :

Dr Tsinjo Tony Rakotoarimalala, Président

M. Rakotondralambo, Examinateur

M. Andoniaina Lahatra Rakotoarimanana, Encadreur Professionnel

Table des matières

Liste des tableaux iii
Liste des figures iv
Glossaire v
Avant-propos 1
L'IT University 1
L'Entreprise d'accueil : OCEANTRADE-SA 2
Remerciements 3
Introduction 4
1 Présentation du projet 5
1.1 Objectifs du projet 5
1.2 Planning de réalisation 5
1.3 Technologies utilisées 7
1.3.1 Protocole OCPP 1.6 en JSON (Open Charge Point Protocol) 7
1.3.2 RabbitMQ 9
1.3.3 Websocket 10
1.3.4 Python (3.12) - FastAPI 10
1.3.5 PostgreSQL v13 12
1.3.6 React.js v18.3.1 13
1.3.7 Git 14
1.3.8 Docker 15
2 Réalisation de l'application 16
2.1 Analyse et conception 16
2.1.1 Analyse de l'existant 16
2.1.2 Conception de l'application 17
2.2 Développement par fonctionnalité ou module 21
2.2.1 Authentification 21
2.2.2 Gestion des utilisateurs 23
2.2.3 Gestion des RFID Utilisateurs 28
2.2.4 Gestion des sessions de recharges 33
2.3 État d'Analyse et Statistiques 39
2.3.1 Analyse de la fréquence d'utilisation des bornes 39

© DAMA Prisca Fehiarisoa 2024

2.3.2 Analyse statistique des habitudes de recharge 39
2.4 Problèmes rencontrés et solutions 40
2.4.1 Problème de coupures de courant lors d'une transaction 41
2.4.2 Problème de configuration réseau dans Docker 41
3 Évaluation du projet et connaissances acquises 42
3.1 Bilan pour l'entreprise 42
3.2 Bilan personnel 42
3.3 Extension et évolution de l'application 42
Conclusion 44
Bibliographie 45

© DAMA Prisca Fehiarisoa 2024

Liste des tableaux

Tableau 1 : Tableau comparatif entre Java et Python 10
Tableau 2 : Tableau comparatif entre FastApi et Django 11
Tableau 3 : Tableau comparatif entre Mysql et Postgresql 13
Tableau 4 : Tableau comparatif entre Angular et React 13
Tableau 5 : Tableau des facteurs de majoration 37

© DAMA Prisca Fehiarisoa 2024

Liste des figures

Figure 1 : Planning Août – Septembre 5
Figure 2 : Planning Septembre – Octobre 6
Figure 3 : Planning Octobre – Novembre 6
Figure 4: Exemple de message OCPP au format JSON 8
Figure 5 : Exemple de message OCPP au format XML 8
Figure 6 : Processus de fonctionnement de RabbitMQ dans le projet 9
Figure 7 : Structure de la base de données du système de gestion de recharge 20
Figure 8 : Interface de connexion 21
Figure 9 : Formulaire de réinitialisation de mot de passe 22
Figure 10 : Format de l'e-mail envoyé pour la réinitialisation de mot de passe 23
Figure 11 : Formulaire de vérification du code envoyé par e-mail 23
Figure 12 : Interface de création d'un nouveau client 24
Figure 13 : Interface pour la liste des clients 25
Figure 14 : Interface pour voir les détails du profile client 25
Figure 15 : interface de modification des Utilisateurs 26
Figure 16 : Interface de confirmation de suppression d'un utilisateur 27
Figure 17 : Interface de filtrage des clients par mois et par années 28
Figure 18 : Interface d'import CSV pour les utilisateurs 28
Figure 19 : Image d'une carte RFID utilisée par l'entreprise pour les recharges 29
Figure 20 : Formulaire de création d'une carte RFID 29
Figure 21 : Liste de toutes les cartes RFID 30
Figure 22 : Formulaire d'importation CSV des cartes RFID 30
Figure 23: Exemple d'un message OCPP : Authorize 32
Figure 24 : Diagramme de Séquence - Processus d'Autorisation RFID 32
Figure 25 : Diagramme de séquence d'un processus de recharge 34
Figure 26 : Liste des transactions effectuées 36
Figure 27 : Bouton de reprise d'une transaction 38
Figure 28 : Détails d'une session de recharge après reprise de la transaction 38
Figure 29 : Analyse de la fréquence d'utilisation des bornes 39
Figure 30 : Statistique de la durée moyenne de recharge 40
Figure 31 : Statistique de l'heure de pointe 40

© DAMA Prisca Fehiarisoa 2024

Glossaire

API (Interface de Programmation d'Applications) : Un ensemble de fonctions et de procédures qui permettent à des applications de communiquer entre elles ou avec des services, facilitant ainsi l'interaction entre les logiciels.

Asynchrone : Un mode d'exécution dans lequel les opérations peuvent être lancées sans attendre le résultat, ce qui permet au programme de continuer son exécution pendant que l'opération s'effectue en arrière-plan.

Backend : La partie d'une application qui gère la logique métier, l'accès aux données, et les opérations serveur, cette partie est généralement invisible pour l'utilisateur final.

Base de données : Un système organisé de stockage des données permettant de structurer, de gérer, et de récupérer de grandes quantités d'informations.

Bibliothèque : Une collection de fonctions, méthodes, ou classes réutilisables qui fournissent des fonctionnalités spécifiques et permettant de simplifier le développement de logiciels.

CSV (Comma-Separated Values) : Un format de fichier texte dans lequel les données sont organisées sous forme de tableau et séparées par des virgules (ou d'autres délimiteurs), permettant un échange facile entre applications.

Diagramme de Gantt : Un diagramme de Gantt est un outil de gestion de projet qui permet de visualiser l'avancement des tâches au fil du temps

DOM (Document Object Model) : Une interface de programmation qui représente la structure d'un document HTML ou XML sous forme d'arbre, permettant aux scripts de modifier dynamiquement le contenu et la structure de la page web.

Framework : Un ensemble d'outils et de bibliothèques qui facilitent le développement d'applications en fournissant une structure et des fonctionnalités de base pour le code.

HTTP (HyperText Transfer Protocol) : Un protocole de communication utilisé sur le web pour le transfert de données entre les navigateurs et les serveurs. Il est sans état, ce qui signifie que chaque requête est indépendante des autres.

© DAMA Prisca Fehiarisoa 2024

JavaScript : Un langage de programmation principalement utilisé pour le développement web afin de créer des pages interactives côté client.

JSON (JavaScript Object Notation) est un format léger de texte utilisé pour structurer et échanger des données. Il est facile à lire pour les humains et simple à traiter pour les machines, souvent utilisé pour transmettre des données entre un serveur et une application web.

JWT (JSON Web Token) : Un format de jeton sécurisé utilisé pour transmettre des informations entre deux parties. Chaque jeton, souvent utilisé pour l'authentification et l'autorisation, est signé pour assurer l'authenticité des données.

Message broker : (courtier de messages) est un composant logiciel ou un middleware qui facilite la communication asynchrone entre différentes applications ou services. Il sert de messager en envoyant, recevant et parfois stockant des messages entre les producteurs (qui envoient des messages) et les consommateurs (qui les reçoivent).

NFC : Near Field Communication est une technologie de communication sans fil qui permet l'échange de données entre les appareils à courte distance, généralement de quelques centimètres. Utilisée principalement dans les paiements mobiles, le transfert de fichiers et l'identification, la technologie NFC fonctionne sur le principe de l'induction électromagnétique pour établir une connexion rapide et sécurisée entre deux dispositifs compatibles

Open source : Un type de logiciel dont le code source est accessible au public, permettant à quiconque de le consulter, de le modifier et de le distribuer.

Protocole TCP (Transmission Control Protocol) : Un protocole de communication qui permet d'assurer la transmission de données de manière fiable entre deux appareils connectés à un réseau.

RFID : Radio Frequency Identification, il s'agit d'un système permettant d'identifier et de suivre des objets à distance en utilisant des ondes radio.

SOAP (Simple Object Access Protocol) est un protocole de communication utilisé pour échanger des informations structurées entre des applications, souvent sur le web. Basé sur XML, il définit des règles strictes pour le format des messages

© DAMA Prisca Fehiarisoa 2024

Token : Une chaîne de caractères qui représente une information d'authentification ou d'autorisation, utilisée pour identifier un utilisateur ou valider des actions dans une application.

TypeScript : Un langage de programmation qui étend JavaScript en ajoutant des types statiques, facilitant la maintenance et la détection des erreurs.

XML (Extensible Markup Language) est un langage de balisage utilisé pour structurer, stocker et transporter des données de manière lisible pour les humains et les machines. Il utilise des balises pour organiser les informations, permettant de représenter des données complexes, et est souvent utilisé dans les échanges de données entre applications et serveurs.

© DAMA Prisca Fehiarisoa 2024

Avant-propos

Le présent mémoire met en évidence les résultats du travail effectué lors de mon stage de fin d'études, en vue de l'obtention du diplôme de Licence en Informatique à l'IT University. Ledit stage a été effectué au Département Informatique de OceanTrade pendant une durée de 3 mois, du mois d'Août au mois de Novembre 2024.
Afin de poser clairement le contexte de ce mémoire, sera présenté succinctement ci-après, d'une part l'IT University et d'autre part mon entreprise d'accueil.

L'IT University

Fondée en 2011, l'IT University (ITU) est une université privée, spécialisée en Informatique, formant les jeunes bacheliers, de préférence scientifiques.

Ci-après les types de formation que IT University propose :

- Formation de trois ans pour l'obtention du diplôme de Licence, au choix de l'étudiant :
    - Licence - Option Développement, Réseaux et Bases de Données ou Web et Design,
    - Licence - Option Graphic Design ou Communication Digitale
- Formation de cinq ans pour l'obtention du diplôme de Master, au choix de l'étudiant :
    - Master MBDS, en coopération avec l'Université Côte d'Azur à Nice Sophia Antipolis – France,
    - Master BIHAR, en coopération avec l'ESTIA du Pays Basque - France

Étant une formation professionnalisante, l'ITU a tissé des liens forts avec ses partenaires industriels, dont l'opérateur convergent TELMA et la plupart des entreprises et institutions du secteur des TIC[1]. Ces partenaires participent effectivement à la formation en agissant comme fournisseur de connexion Internet à haut débit, avec l'envoi de conférenciers ou en accueillant des étudiants en stage. Beaucoup de ces partenaires recrutent aussi les nouveaux diplômés dès leur sortie d'École.

[1] TIC : Technologies de l'Information et de la Communication

D’autre part, le corps enseignant de l’ITU est constitué intégralement de spécialistes de très haut niveau et obligatoirement actifs professionnellement dans leurs domaines respectifs.

L’Entreprise d’accueil : OCEANTRADE-SA

OCEANTRADE-SA a une histoire riche et en constante évolution. À l'origine, elle s'appelait HIRIMEX, une petite entreprise familiale implantée à Alarobia, et spécialisée dans la fabrication de textile et la confection de vêtements. À une époque où Madagascar traversait de grandes difficultés d'importation, HIRIMEX a su saisir cette opportunité pour devenir un acteur clé du marché local.

Sous la direction de M. Razaly Hiridjee, l'entreprise a changé de nom pour devenir HARY, puis Géo Textile. En 1975, un tournant majeur est survenu avec l'acquisition d'un terrain stratégique appartenant à l'Entreprise de Construction Automobile de Madagascar. Ce nouveau siège à Andraharo a ouvert la voie à l'expansion de l'entreprise, qui a ensuite formé une alliance dans le secteur automobile, notamment dans la vente de pièces détachées et la fabrication de voitures.

Dans les années 1980, la société s'est associée à Japan Motors (aujourd'hui CONTINENTAL AUTO), élargissant ses activités dans le domaine automobile, notamment dans la vente de marques japonaises comme ISUZU et MAZDA. En 2002, avec la montée en flèche de la friperie, l'activité textile a progressivement pris fin, laissant l'automobile devenir le cœur du métier d'OCEANTRADE.

Depuis, l'entreprise s'est imposée comme l'une des concessionnaires automobiles les plus respectées à Madagascar, notamment en se démarquant avec la commercialisation de marques chinoises très prisées par les clients malgaches. En 2015, OCEANTRADE a rejoint le groupe VISEO, aux côtés de trois autres entreprises du même fondateur : CONTINENTAL AUTO, IZYRENT et HAVAMAD.

Aujourd’hui, OCEANTRADE poursuit sa croissance dans le commerce et la fourniture de services, s'adaptant aux défis d’un marché en constante évolution. Fidèle à sa philosophie d’entreprise, OCEANTRADE place le capital humain au cœur de son développement, veillant à valoriser et soutenir ses collaborateurs tout en cultivant des relations de confiance avec ses clients.

© DAMA Prisca Fehiarisoa 2024

Remerciements

Avant toutes choses, je souhaiterai exprimer ma gratitude envers toutes les personnes qui ont contribué, de près comme de loin, à la réalisation du présent mémoire.

Je tiens particulièrement à remercier :

- Les membres du Jury : **Docteur Tsinjo Tony Rakotoarimalala** et **Monsieur Rakotondralambo** pour le temps qu'ils ont consacré à l'évaluation du présent mémoire et pour leurs observations constructives, qui ont permis de l'enrichir.
- **Monsieur Andoniaina Lahatra Rakotoarimanana**, mon encadreur professionnel pour sa disponibilité, son accompagnement et son accueil pendant mon stage à OceanTrade.
- Le corps enseignant et le personnel administratif de l'IT University pour leur accompagnement et l'excellence de leur enseignement.
- Mes amis et collègues de stage pour leurs encouragements et leur soutien moral durant cette expérience.
- Ma famille pour leur soutien inconditionnel, tant moral que matériel pour mener à bien ce projet.

A vous tous, un grand Merci.

© DAMA Prisca Fehiarisoa 2024

Introduction

L’essentiel du travail que nous avons réalisé a porté sur le thème « Logiciels de gestion des bornes de recharge des véhicules électriques, » visant à faciliter la gestion des bornes de recharges mises à la disposition des clients et la gestion des sessions des recharges.

Imaginez un Madagascar où dans les rues, on ne voit que des véhicules électriques silencieux, où les gaz d’échappement font partie d’une histoire au passé et où l'air est pur. Ce rêve de mobilité durable est en train de devenir peu à peu une réalité avec l’essor des véhicules électriques qui représentent une alternative propre aux véhicules thermiques. Cependant, cette évolution dépendra largement de notre capacité à mettre en place un réseau de bornes de recharge accessibles et fiables. À Madagascar, bien que certaines entreprises privées commencent à faire bouger les lignes, l'infrastructure actuelle reste insuffisante pour soutenir cette dynamique.

C'est dans ce contexte que le projet d'OceanTrade prend toute son importance. En s'inscrivant dans la dynamique de développement de solutions informatiques innovantes, ce projet vise à optimiser la gestion des bornes de recharge et à accompagner la transition énergétique locale. En intégrant des modules tels que la gestion des utilisateurs via des cartes RFID, la gestion des bornes de recharge, et le suivi des sessions de recharges, OceanTrade aspire à offrir une expérience utilisateur fluide tout en garantissant une facturation précise.

Au cours de mon stage, j’ai contribué à plusieurs aspects clés du développement du logiciel. Les résultats de mon travail incluent les éléments suivants :

- Le développement d'une authentification via JWT pour sécuriser l'accès à l'application,
- La gestion des utilisateurs et des cartes RFID associées,
- La mise en place d’un système de supervision et de gestion des sessions de recharge,
- L’implémentation d'un module de gestion de la tarification des sessions de recharge.

Ainsi, dans un premier temps, nous présenterons les détails du projet ; ensuite nous examinerons sa réalisation ; et enfin nous procéderons à une évaluation du projet et des connaissances acquises.

© DAMA Prisca Fehiarisoa 2024

1 Présentation du projet

Comme nous l'avions mentionné précédemment, ce projet figure parmi la réalisation du leitmotiv d'OceanTrade, celui de placer le capital humain au centre de ses activités et anticiper les besoins du futur, en créant le système qui permettra de gérer les bornes de recharges pour les véhicules électriques.

1.1 Objectifs du projet

L'objectif principal de ce projet est ainsi de créer un système de Gestion des bornes de recharges des véhicules électriques, ce qui va permettre à l'entreprise de superviser efficacement les sessions de recharges et la tarification, ainsi que de gérer les utilisateurs et les bornes de recharges.

Pour être plus précis, ce projet vise à :

- Créer une communication en temps réel entre les bornes de recharges et le centre de contrôle,
- Faciliter l'accès des utilisateurs à la borne de recharge grâce à la carte RFID,
- Superviser les sessions de recharges,
- Fournir des rapports d'analyses sur l'utilisation des bornes de recharges.

1.2 Planning de réalisation

La réalisation de mes tâches au cours du développement de ce projet a été planifié suivant le diagramme de Gantt selon ce qui est présenté ci-après :

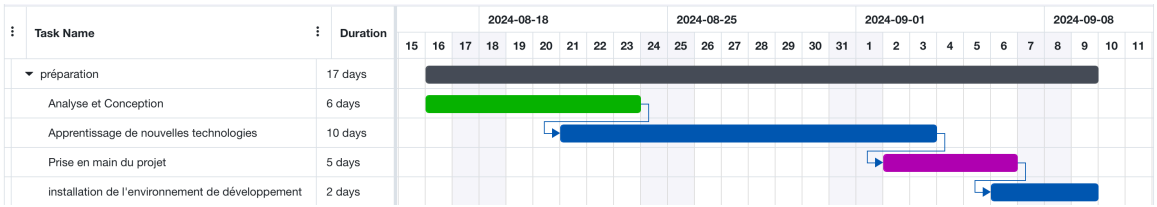


Figure 1 : Planning Août – Septembre

Comme le montre la Figure 1, la phase de Préparation, incluant la prise en main du projet, l'apprentissage des technologies, l'installation de l'environnement de développement, et l'analyse et conception, s'est déroulée sur tout le mois d'août et les dix premiers jours de

© DAMA Prisca Fehiarisoa 2024

septembre, totalisant 17 jours. Ayant débuté mon stage près d'un mois après les autres, je n'ai pu consacrer que 05 jours à cette phase, pour rattraper le rythme et répondre aux exigences du projet.

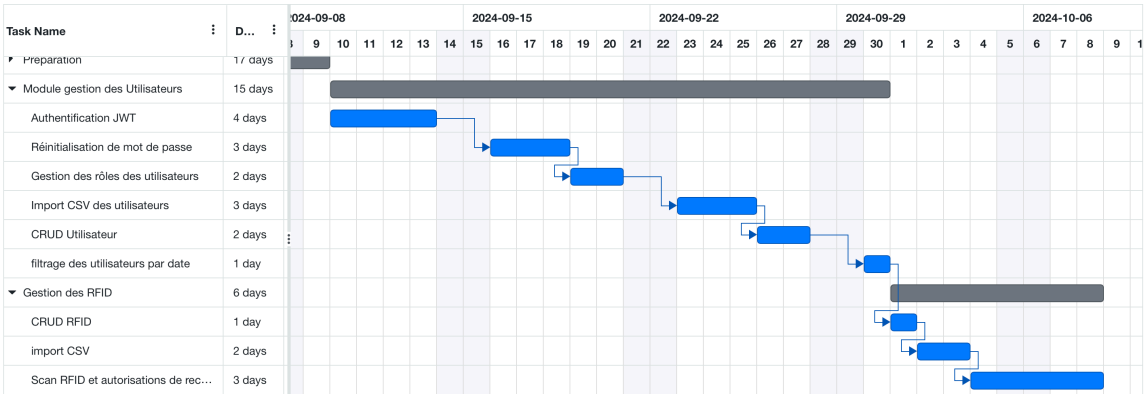


Figure 2 : Planning Septembre – Octobre

La deuxième figure montre le calendrier de démarrage du projet, qui s'est étendu de septembre à début octobre. Au cours de cette période, les travaux se sont concentrés sur le développement des modules clés, notamment la gestion des utilisateurs et des cartes RFID. Chaque étape, de l'authentification JWT à la gestion des rôles des utilisateurs, ainsi que la configuration des cartes RFID, a été réalisée de manière progressive conformément au calendrier établi.

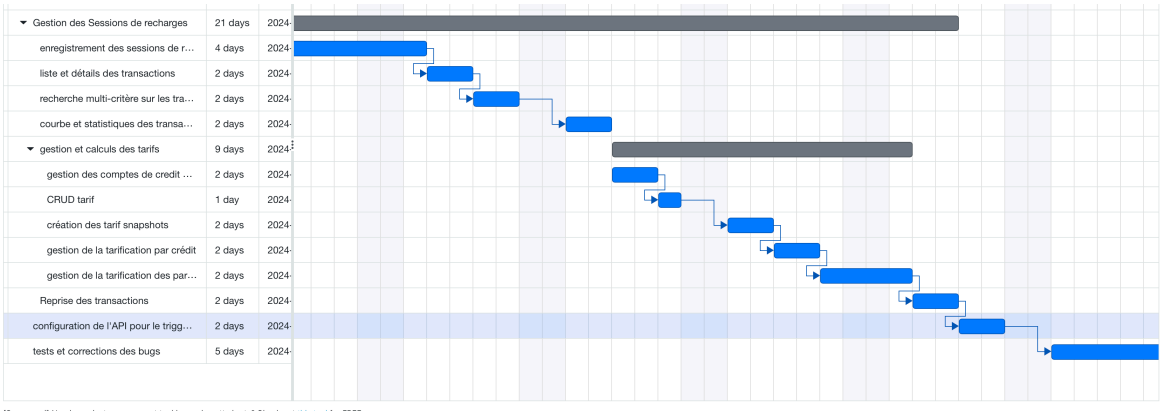


Figure 3 : Planning Octobre – Novembre

La Figure 3 représente la dernière phase du planning, couvrant la réalisation de la partie centrale du projet, à savoir le développement du module de gestion des recharges de véhicules, ainsi que les tests associés. Cette étape, d'une durée de 29 jours, s'est déroulée entre octobre et novembre. La partie tarification a été particulièrement complexe, nécessitant

© DAMA Prisca Fehiarisoa 2024

une attention particulière en raison des nombreux paramètres à prendre en compte pour offrir une solution flexible et précise.

1.3 Technologies utilisées

La réalisation de ce projet a nécessité l'intégration de huit technologies distinctes pour en assurer le bon fonctionnement. Parmi elles, on retrouve le protocole OCPP version 1.6 JSON, l'outil de messagerie RabbitMQ, le protocole réseau WebSocket, le langage de programmation Python (3.12) avec FastAPI, le système de gestion de base de données PostgreSQL, la bibliothèque JavaScript React, le système de contrôle de version Git, ainsi que l'outil de conteneurisation Docker.

1.3.1 Protocole OCPP 1.6 en JSON (Open Charge Point Protocol)

Le Protocole OCPP ou Open Charge Point Protocol, version 1.6 JSON est un protocole standard et *open source* qui permet la communication entre les points de recharge des véhicules électriques et le centre de contrôle. Il définit comment les bornes de recharge et le backend échangent des informations, en envoyant des messages pour gérer divers aspects du processus de recharge. Par exemple, les messages *« StartTransaction »* ou *« StopTransaction »* démarrent et arrêtent une session de recharge, tandis que le message « *Authorize* » gère les autorisations de recharges. Les messages tels que « HeartBeat » et « MeterValues » sont utilisés pour la surveillance et le suivi en temps réel.

Les messages de la version 1.6 de OCPP peuvent être présentés sous deux formats :

- JSON (Javascript Object Notation) : Un format de texte simple qui utilise des paires de clé-valeurs. Il utilise des Websockets pour établir une communication bidirectionnelle en temps réel afin de transporter les messages.

  Voici un exemple de message OCPP au format JSON :

© DAMA Prisca Fehiarisoa 2024

```json
{
    "messageTypeId": 2,
    "messageId": "123456",
    "action": "StartTransaction",
    "payload": {
        "connectorId": 1,
        "idTag": "ABC123"
    }
}
```

Figure 4: Exemple de message OCPP au format JSON

- SOAP (Simple Object Access Protocol) : Un format basé sur XML mais qui est plus complexe que le JSON. Les messages utilisent les protocoles http et https pour leur transport.

  Voici un exemple de Message OCPP au format XML :

```xml
<soapenv:Envelope
        xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
        xmlns:ocpp="urn://Ocpp/Cs/2012/06/">
    <soapenv:Header/>
    <soapenv:Body>
        <ocpp:startTransactionRequest>
            <connectorId>1</connectorId>
            <idTag>ABC123</idTag>
        </ocpp:startTransactionRequest>
    </soapenv:Body>
</soapenv:Envelope>
```

Figure 5 : Exemple de message OCPP au format XML

Dans le cadre de ce projet, nous utilisons la version 1.6 de l'OCPP, qui prend en charge le format JSON en raison de sa simplicité, de sa lisibilité, de sa compatibilité avec les Websockets et de sa facilité d'intégration avec les APIs. Ces caractéristiques font de JSON un choix idéal pour gérer les échanges d'informations entre les bornes de recharge et le centre de contrôle.

© DAMA Prisca Fehiarisoa 2024

1.3.2 RabbitMQ

RabbitMQ est un outil qui sert d'intermédiaire pour transmettre des messages entre différents systèmes, comme les bornes de recharge et le centre de contrôle. Il gère les messages de manière asynchrone en les plaçant dans une file d'attente jusqu'à ce qu'ils soient traités par le backend, permettant ainsi aux deux systèmes de fonctionner de manière indépendante.

Voici un schéma qui montre comment fonctionne RabbitMQ :

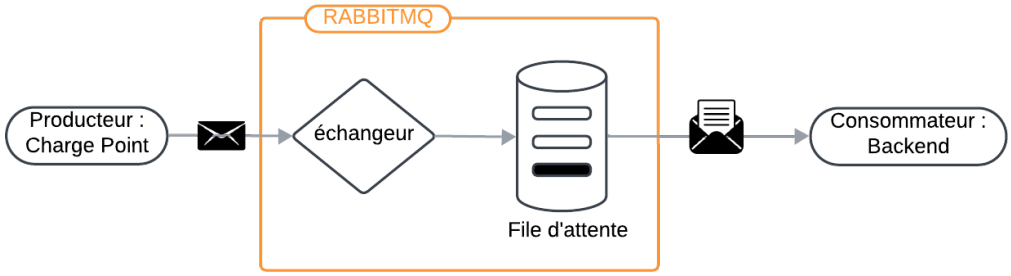


Figure 6 : Processus de fonctionnement de RabbitMQ dans le projet

Fonctionnement de RabbitMQ :

1. Producteur : Envoie des messages à un échangeur, qui peut utiliser une clé de routage pour diriger le message.
2. Échangeur : Reçoit le message et l'achemine vers la file d'attente appropriée selon le type d'échange configuré :
   - *Fanout exchange*: Envoie le message à toutes les files d'attente.
   - *Direct exchange*: Envoie le message à la file d'attente correspondant à sa clé de routage.
   - *Topic exchange*: Envoie le message à la file d'attente avec une clé de routage similaire.
   - *Header exchange* : Utilise l'en-tête du message pour le diriger vers la bonne file.
1. File d'attente : Stocke les messages jusqu'à ce qu'un consommateur (responsable du traitement) les récupère.

© DAMA Prisca Fehiarisoa 2024

1.3.3 Websocket

Un Websocket est un protocole réseau basé sur le protocole TCP qui permet de créer une connexion, favorisant l'envoi et la réception en continu des données entre un client et un serveur ou un serveur avec un autre serveur. Contrairement au protocole HTTP qui fonctionne en mode « requête-réponse » où le client envoie une requête et le serveur renvoie une réponse puis la connexion se ferme, un websocket permet de maintenir une connexion ouverte pour que l'échange d'information puisse se faire en temps réel.

1.3.4 Python (3.12) - FastAPI

- **Python 3.12**

Pour ce projet, il était crucial de choisir un langage de programmation capable de gérer efficacement les communications entre les bornes de recharges et le centre de contrôle, ainsi que de s'adapter à la croissance du système. Deux langages se sont présentés comme des options viables : **Java** et **Python**. Chacun d'eux possède ses propres avantages et inconvénients, notamment en termes de performance, de gestion des tâches asynchrones et de facilité de développement. Voici un tableau qui montre quelques-unes des fonctionnalités proposées par Java et Python.

Tableau 1 : Tableau comparatif entre Java et Python

Critères	**Java**	**Python**
Langage	**Compilé** : le code source est compilé en un autre langage avant son exécution	**Interprété** : le code source est exécuté directement
Flexibilité	Flexible mais nécessite des étapes de compilation et de configurations supplémentaires	Très flexible car l'exécution du programme ne nécessite pas de phase de compilation
Développement	Développement plus lent mais meilleure performance à l'exécution	Plus rapide à développer mais plus lente à l'exécution
Support du protocole OCPP	Bibliothèque : **OCPP-J**	Bibliothèque : **OCPP-python**

© DAMA Prisca Fehiarisoa 2024

Support asynchrone	Gère les fonctions asynchrones mais nécessite une configuration plus complexe	Possède un support asynchrone natif (asyncio)
Performance	Performant pour les applications lourds	Plus léger et rapide à configurer, idéal pour les applications simples ou les services REST

Après comparaison, Python s'est révélé être le choix idéal. En tant que langage interprété, il est flexible et permet un développement rapide sans les étapes de compilation de Java, ce qui réduit le temps de développement, essentiel pour les applications simples et les services REST. Avec sa bibliothèque native *asyncio*, Python gère facilement les fonctions asynchrones, assurant une exécution fluide en temps réel. De plus, étant donné que l'entreprise utilise Odoo, basé sur Python, cela facilite grandement l'intégration avec les systèmes existants, un atout décisif dans le choix de Python.

- **FastAPI**

FastAPI est un Framework web moderne et performant, conçu pour la création des API, interfaces de programmation d'applications utilisant Python. Il joue un rôle clé dans notre projet en facilitant la création et la gestion des API, nécessaires pour assurer la communication entre les bornes de recharges et le centre de contrôle. FastAPI présente un avantage important du fait qu'il prend en charge les Websocket, ce qui est essentiel puisque le protocole OCPP repose sur Websocket afin de permettre une communication en temps réel entre les différentes parties du système.

Afin de justifier le choix de FastAPI, une analyse comparative sur plusieurs critères clés ont été faites. Ci-dessous un tableau qui résume cette comparaison :

Tableau 2 : Tableau comparatif entre FastApi et Django

	FastAPI	**DJANGO**
Langage	Python	Python

© DAMA Prisca Fehiarisoa 2024

Architecture	Basée sur ASGI (Asynchronous Server Gateway Interface) qui est Asynchrone	Basée sur WSGI (Web Server Gateway Interface) qui est Synchrone
Prise en charge des Websockets	Supporte nativement les Websockets	Disponible en utilisant DJANGO CHANNEL
Gestion des API REST	Intégrée nativement	Nécessite l'utilisation de *Django Rest Framework*
Légèreté	Plus léger et facile à maitriser pour les projets utilisant des API ou microservices	Plus complet et imposant
Scalabilité	Excellente pour les microservices et les architectures distribuées	Plus adaptée pour les applications monolithiques

FastAPI a été choisi pour ses nombreux atouts. Son architecture asynchrone ASGI optimise les performances en permettant des actions indépendantes. De plus, il supporte les Websockets et gère nativement les API REST. Léger, facile à prendre en main et hautement scalable, FastAPI est parfaitement adapté aux projets d'API et de microservices, faisant de lui l'outil idéal pour ce projet.

Ainsi, en accord avec les exigences de l'entreprise, Python a été retenu pour le développement de l'application, permettant d'exploiter pleinement les fonctionnalités de FastAPI et d'assurer une intégration fluide avec les autres composants du projet.

1.3.5 PostgreSQL v13

PostgreSQL est un système de gestion de base de données open source, robuste, performant et extensible. Sa capacité à gérer un grand nombre de données et sa fiabilité ont pesé dans le choix de l'utilisation de PostgreSQL dans le projet.

Voici le tableau comparatif, qui a motivé le choix de l'utilisation de PostgreSQL pour le projet :

© DAMA Prisca Fehiarisoa 2024

Tableau 3 : Tableau comparatif entre Mysql et Postgresql

Critères	**PostgreSQL**	**MySQL**
Licence	Utilise la « licence PostgreSQL » qui est open source	Utilise la licence GPL (General Public License) qui est Open source ou bien utilise une licence Commerciale qui permet une utilisation sans les contraintes de la GPL
Performances	Très performant pour les requêtes complexes et lourdes	Moins performant pour les requêtes complexes, plus optimisé pour des requêtes simples.
Extensibilité	Très extensible, il permet d'ajouter des types de données, des opérateurs, des fonctions, et des langages supplémentaires.	Extensibilité limitée, surtout axée sur les fonctionnalités de base des bases de données relationnelles.

Ainsi, PostgreSQL s'est révélé être un choix stratégique pour ce projet, garantissant robustesse et flexibilité pour répondre aux besoins spécifiques de l'application.

1.3.6 React.js v18.3.1

React est une bibliothèque JavaScript axée sur la création d'interfaces utilisateurs réactives et dynamiques. Elle permet de construire des composants réutilisables qui facilitent le développement d'applications complexes.

Avant de faire notre choix, nous avons réalisé une étude comparative approfondie, dont le tableau ci-dessous illustre les principales différences entre React et Angular, deux technologies populaires pour le développement d'interfaces utilisateurs.

Tableau 4 : Tableau comparatif entre Angular et React

Critères	**React**	**Angular**

© DAMA Prisca Fehiarisoa 2024

Type de Framework	Bibliothèque javascript centrée sur l'interface utilisateur	Framework complet basé sur Typescript
Langage principal utilisé	JavaScript ou TypeScript	Typescript
Optimisation des Performances	React utilise un DOM virtuel, qui est une représentation plus légère du DOM réel. Cela améliore l'efficacité en ne mettant à jour que les parties nécessaires de l'interface utilisateur, minimisant ainsi les opérations coûteuses sur le DOM.	Angular optimise les performances en compilant le code dans le navigateur et en éliminant le code inutilisé
Architecture	Basée sur l'utilisation de composants réutilisables	Basée sur un système modulaire
Flexibilité	Flexible, mais nécessite l'utilisation de bibliothèques externes pour les fonctionnalités comme le routage ou les requêtes http	Suit une structure stricte ce qui peut limiter la flexibilité par rapport à des solutions plus légères

Ainsi, par rapport à la comparaison des deux bibliothèques, le choix de React pour ce projet a été motivé par sa flexibilité et sa simplicité d'utilisation, étant donné qu'il est centre sur l'utilisateur, et, bien que tout comme Angular, elle utilise Typescript, elle a ce plus de l'utilisation de JavaScript. En favorisant une approche basée sur des composants réutilisables, React permet de développer des interfaces dynamiques et performantes tout en facilitant la maintenance du code.

1.3.7 Git

Git est un système de contrôle de version utilisé dans le cadre du développement logiciel afin de gérer les modifications effectuées sur le code source. Il permet aux développeurs de collaborer en suivant les changements effectués par les autres développeurs, en travaillant sur des versions séparées du code par l'intermédiaire des branches. Il enregistre

© DAMA Prisca Fehiarisoa 2024

également un historique des modifications permettant aux développeurs de revenir facilement sur les modifications ultérieures du code en cas de besoin.

1.3.8 Docker

Docker est un outil qui permet de créer des conteneurs dans lesquels on peut exécuter et déployer des applications. Un conteneur est comme une boite dans laquelle on peut emballer tout ce dont une application a besoin comme le code, les configurations et bien d'autres outils encore pour fonctionner. Cela permet de s'assurer que l'application fonctionne de la même manière peu importe où elle est exécutée que ce soit dans un ordinateur, un serveur ou dans le cloud.

Chaque application ou service est souvent isolé dans son propre conteneur, permettant ainsi d'éviter les conflits, mais le plus de Docker c'est qu'il permet de contenir plusieurs services dans un même conteneur si nécessaire. Contrairement aux machines virtuelles, les conteneurs partagent le noyau du système d'exploitation hôte, ce qui leur permet d'être plus légers et d'utiliser moins de ressources. Cela facilite l'exécution simultanée de plusieurs applications sur un même serveur tout en maintenant des performances élevées.

En somme, ces outils technologiques ont été choisis par rapport aux choix et exigences de l'entreprise, mais également par rapport à leur flexibilité et leur performance.

© DAMA Prisca Fehiarisoa 2024

2 Réalisation de l'application

La réalisation de cette application a été motivée par plusieurs choses, entres autres le besoin de toujours innover pour mieux répondre aux besoins du marché mais aussi pour avoir une vue d'ensemble de l'évolution de la gestion des bornes de recharge. Ainsi, avant de réaliser l'application, l'analyse de l'existant était une nécessité pour pouvoir analyser le besoin de conception, et ensuite procéder au développement par fonctionnalité ou module.

2.1 Analyse et conception

Dans cette section, nous nous concentrerons sur l'analyse et la conception du système de gestion des bornes de recharge, dont l'objectif principal est d'optimiser la gestion des infrastructures de recharge détenues par notre entreprise.

2.1.1 Analyse de l'existant

Avant de procéder à la conception de l'application, il est essentiel d'analyser le système existant, les infrastructures pour effectuer une recharge et les besoins de l'entreprise

Pour l'instant, OceanTrade a installé une seule borne de recharge, mais envisage d'en ajouter davantage afin de renforcer son engagement, son soutien pour les initiatives écologiques à Madagascar et aussi pour améliorer la satisfaction client existant et en attirer de nouveaux. Voici donc succinctement ce qui a amené au besoin de conception du projet :

- État de l'infrastructure actuelle :
    - Une seule borne installée
    - Les sessions de recharges ne sont pas enregistrées
    - Aucun contrôle d'accès pour les utilisateurs, les clients n'est mise en place.
- Identification des besoins de l'entreprise :
    - Absence du contrôle et de l'authentification des utilisateurs : vu l'infrastructure actuelle, il y a un manque de sécurité et de supervision, puisque tout le monde peut avoir accès à l'utilisation la borne, ce qui représente un grand risque pour l'entreprise en termes de rentabilité, de durabilité des bornes installées, mais également de détection des utilisateurs en cas de vandalisme.

© DAMA Prisca Fehiarisoa 2024

    - Suivi des sessions et des transactions : il est important de gérer les sessions de recharges, de générer des rapports de consommations et d'identifier les utilisateurs ayant initié la recharge.
    - Scalabilité : Avec plusieurs bornes prévues, un système de gestion centralisé est nécessaire pour suivre l'activité de chaque borne et de gérer efficacement leur usage.
    - Maintenance : sans suivi de l'état des bornes, il est difficile de détecter les besoins de maintenance.

- Solutions envisagées :
    - Utilisation de cartes RFID pour restreindre l'accès aux bornes de recharge aux utilisateurs autorisés.
    - Développement d'une interface pour les administrateurs afin de suivre en temps réel les opérations de recharge, obtenir des rapports détaillés et de superviser les utilisateurs et les bornes.

L'analyse de l'infrastructure actuelle révèle des lacunes significatives dans la gestion des bornes de recharge électriques, notamment l'absence de contrôle d'accès et de suivi des sessions. Il est donc essentiel d'implémenter des solutions comme l'authentification par carte RFID lors des recharges et un système centralisé de suivi. Ces améliorations seront déterminantes dans la sécurité et l'efficacité opérationnelle des bornes de recharges de l'entreprise.

2.1.2 Conception de l'application

Pour répondre efficacement aux exigences du projet, la phase de conception est essentielle. Après avoir analysé les besoins et discuté des solutions possibles, la conception peut se faire. Cette étape vise à établir une feuille de route claire pour un travail efficace. S'engager directement dans le développement sans consacrer le temps nécessaire à une conception approfondie pourrait entraîner des problèmes sur le long terme.

© DAMA Prisca Fehiarisoa 2024

2.1.2.1 Architecture de l'application

Afin d'assurer une gestion optimale et une communication fluide entre les différents composants du logiciel, nous avons opté pour une architecture n-tiers. Cette architecture comprend plusieurs couches distinctes :

- Frontend : développé en React, il permet aux utilisateurs de gérer les bornes de recharges et les sessions de recharge via une interface graphique.
- Backend : utilisant FastAPI pour la logique métier et la gestion des interactions avec les bornes.
- RabbitMq : un message broker Assurant une communication asynchrone avec les bornes de recharge grâce au protocole OCPP.
- PostgreSQL : Gérant la persistance des données critiques, telles que les informations de session et les utilisateurs.

Cette structure modulaire répond efficacement aux besoins de scalabilité et de performance du système tout en assurant une expérience utilisateur fluide.

2.1.2.2 Conception des modules et structure de la base de données

Ainsi, pour la conception du module d'authentification et le module gestion des utilisateurs, nous avons créé :

- Six (06) tables côté base de données
- Neuf (09) écrans côté affichage
- Un format d'email pour la réinitialisation de mot de passe

Pour la conception du module concernant les RFID, nous avons créé :

- Trois (3) tables
- Trois (03) écrans

Pour celui du module concernant les transactions de recharges, nous avions prévu :

- Trois (03) tables
- Quatre (04) écrans

Pour la partie tarification, nous avions décidé de créer :

- Deux (02) tables
- Trois (03) écrans

© DAMA Prisca Fehiarisoa 2024

Après avoir détaillé les exigences de chaque module, nous avons conçu la structure de la base de données pour répondre aux besoins fonctionnels du projet. La figure ci-dessous montre les tables créées pour chaque module, ainsi que leurs relations. Cette structure de base de données comprend les tables principales pour chaque module :

- Module d'authentification et gestion des utilisateurs : inclut les tables *usergroup, user_table, subscription*, etc., qui gèrent les informations des utilisateurs et les abonnements.
- Module RFID : contient les tables *tag, usercredit,* et *rfid_usage_history*, permettant de suivre l'usage des tags RFID et les crédits associés.
- Module des transactions de recharge : comporte les tables *transaction, session, historique_session*, etc., pour suivre les sessions de recharge et les transactions associées.
- Module de tarification : inclut les tables *tariff* et *tariffsnapshot*, qui définissent les tarifs en fonction des plages horaires et des types d'énergie.

© DAMA Prisca Fehiarisoa 2024

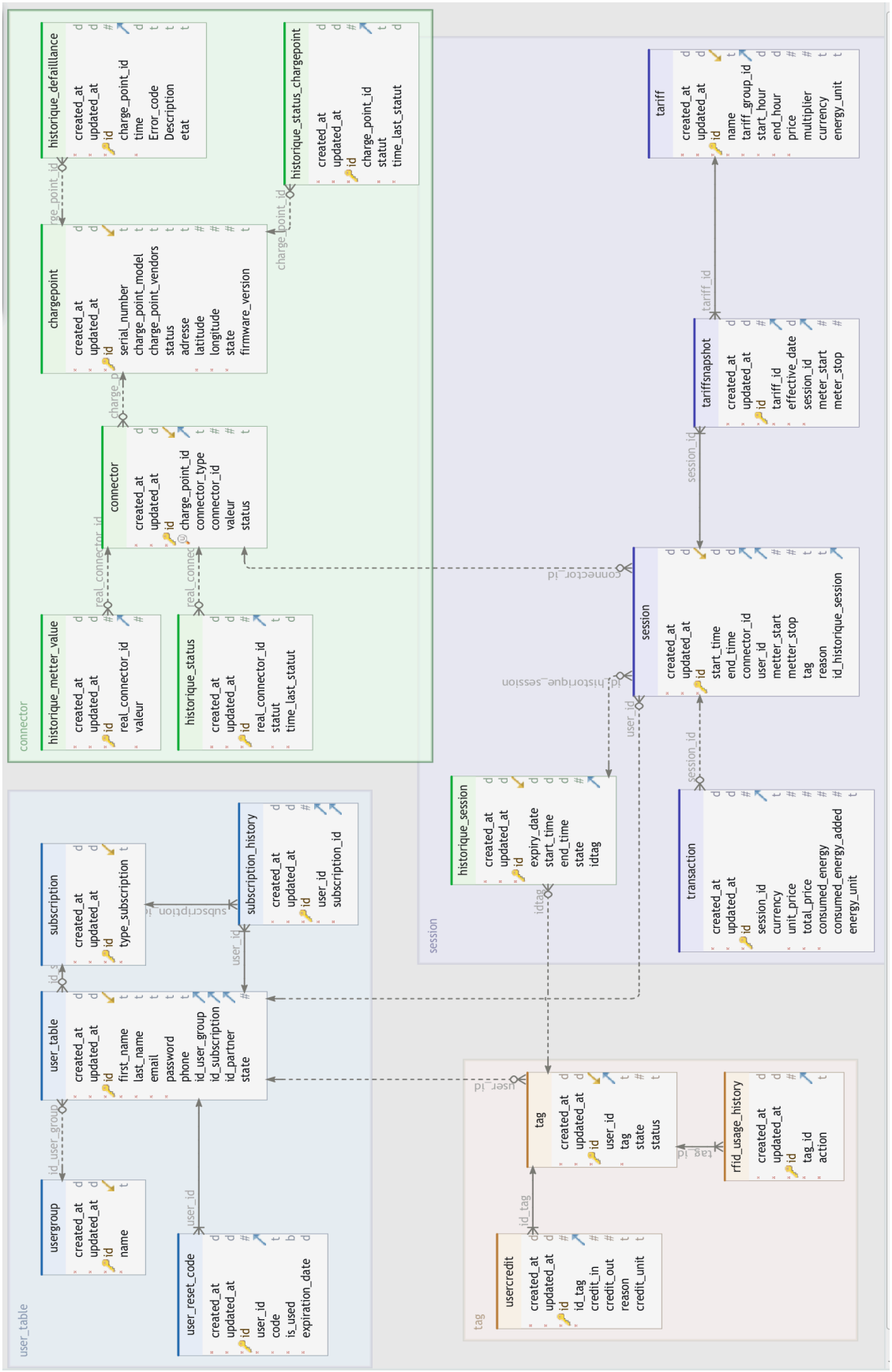


Figure 7 : Structure de la base de données du système de gestion de recharge

© DAMA Prisca Fehiarisoa 2024

2.2 Développement par fonctionnalité ou module

2.2.1 Authentification

Ce module permet de s'assurer que seuls les administrateurs, et les autres membres du personnel autorisés peuvent accéder aux différentes fonctionnalités de l'application. Son rôle est donc de sécuriser l'accès en vérifiant l'identité des utilisateurs.

Il englobe les fonctionnalités suivantes :

2.2.1.1 La connexion

Cette fonctionnalité permet à l'utilisateur d'entrer son mot de passe et son email. Le serveur procède ensuite à une vérification des identifiants avant d'autoriser la connexion. Si les informations sont correctes, le serveur génère un JSON Web Token ou JWT, un jeton qui permet à l'utilisateur d'accéder à l'application sans avoir à ressaisir ses identifiants pendant 30 minutes. Si les informations sont incorrectes, un message d'erreur est renvoyé à l'utilisateur. La figure suivante montre un aperçu de l'interface de connexion :



Figure 8 : Interface de connexion

Le JWT contient des informations sur l'utilisateur, nécessaires pour l'identifier. De plus, il est signé ce qui signifie qu'il est protégé contre les modifications non autorisées. Pour ce projet, nous utilisons l'algorithme « ES256 » pour la signature des JWT.

© DAMA Prisca Fehiarisoa 2024

Pour renforcer la sécurité, les mots de passes sont hachés à l'aide d'un algorithme « bcrypt » ce qui les rend illisibles. Les jetons de connexions ont également une durée de validité, après laquelle l'utilisateur doit se reconnecter.

2.2.1.2 La réinitialisation du mot de passe

Cette fonctionnalité permet à l'utilisateur de modifier son mot de passe. Pour cela, la bibliothèque FastAPI-Mail a été utilisée pour pouvoir envoyer des e-mails de manière non synchronisée.

Lorsque l'utilisateur souhaite réinitialiser son mot de passe, il peut soumettre une demande en remplissant un formulaire où il donne son adresse e-mail. Le système génère alors un code d'accès à six (06) chiffres qui est envoyé à l'adresse e-mail de l'utilisateur. Une fois le code reçu, l'utilisateur peut l'entrer sur une page destinée à cet effet et peut ensuite changer son mot de passe. Le code doit être utilisé dans un délai de 10 minutes.

Les figures ci-après montre un visuel de de l'écran à chaque étape du processus de réinitialisation du mot de passe.

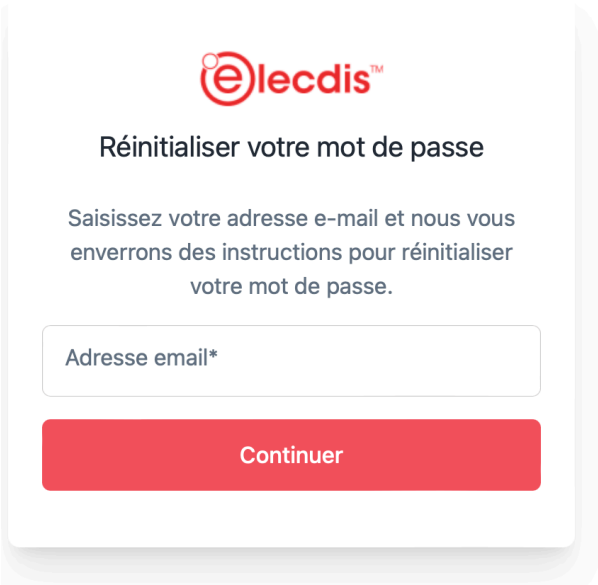


Figure 9 : Formulaire de réinitialisation de mot de passe

© DAMA Prisca Fehiarisoa 2024

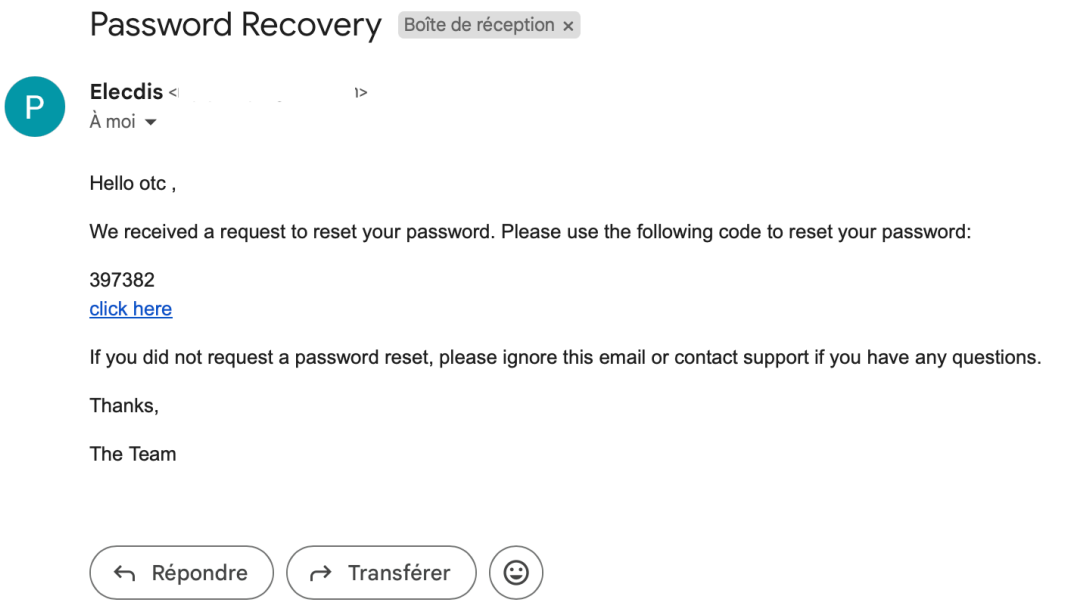


Figure 10 : Format de l'e-mail envoyé pour la réinitialisation de mot de passe

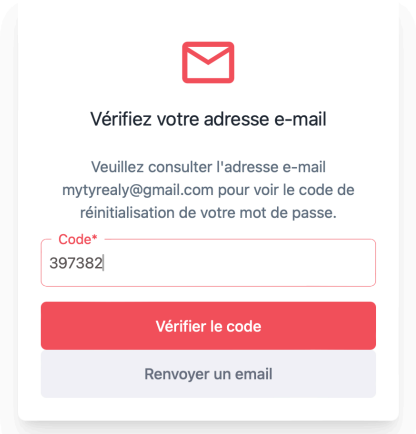


Figure 11 : Formulaire de vérification du code envoyé par e-mail

2.2.2 Gestion des utilisateurs

La gestion des utilisateurs est un aspect crucial pour garantir une interaction fluide avec l'application. Elle permet de contrôler l'accès, de gérer les différents profils et d'assurer un suivi précis des informations des utilisateurs. Dans cette section, nous détaillerons les processus et les fonctionnalités associées à la gestion des utilisateurs dans l'application.

© DAMA Prisca Fehiarisoa 2024

2.2.2.1 CRUD Utilisateurs

L'une des principales opérations de gestion des utilisateurs repose sur la possibilité de réaliser des actions CRUD (Créer, Lire, Mettre à jour, Supprimer) sur leurs profils. Cette fonctionnalité permet à l'administrateur de maintenir à jour la base de données des utilisateurs, assurant ainsi une gestion dynamique et flexible des informations. Nous allons maintenant examiner en détail comment ces opérations sont implémentées dans le système.

- **Création des Utilisateurs** :

La création des utilisateurs est essentielle pour enregistrer les clients set leur permettre d'utiliser les bornes de recharge, notament via des cartes d'acces comme la carte RFID. Cette fonctionnalité permet d'ajouter de nouveaux utilisateurs au système.

Voici l'interface montrant le formulaire de création d'un client :

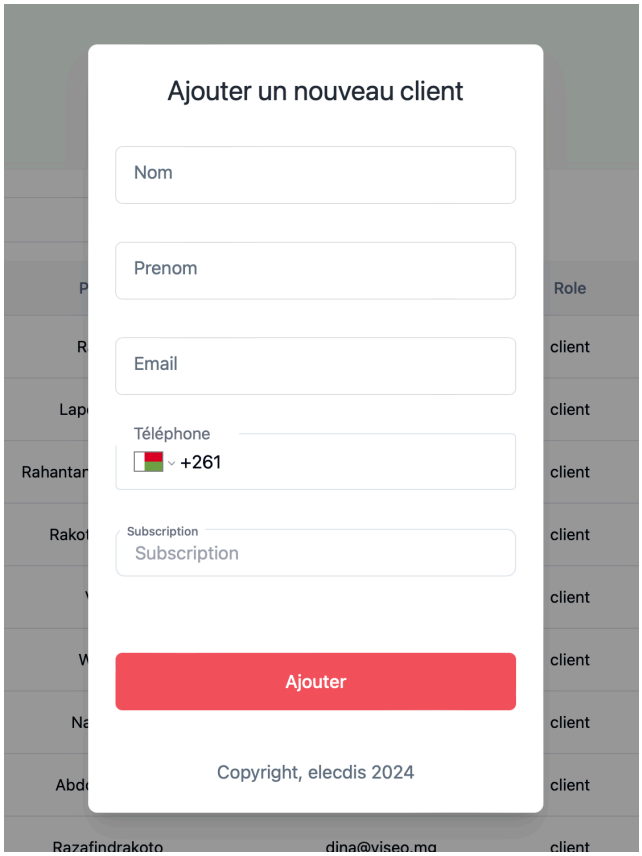


Figure 12 : Interface de création d'un nouveau client

- **Consultation des informations des utilisateurs** :

Les administrateurs peuvent consulter les informations des utilisateurs pour vérifier leurs données ou effectuer des recherches.

© DAMA Prisca Fehiarisoa 2024

Voici l'interface pour afficher la liste des clients :

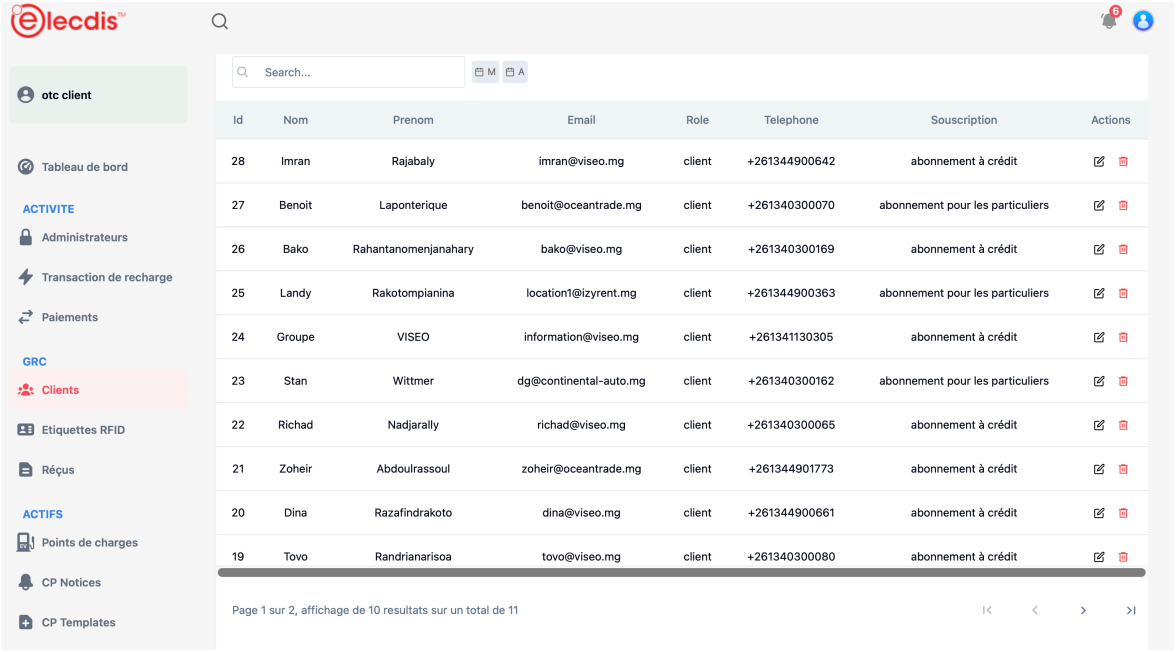


Figure 13 : Interface pour la liste des clients

Voici l'interface pour afficher les détails du profil d'un client. On peut y voir la liste de ses cartes RFID, la liste des sessions qu'il a effectuée et ses informations personnelles.

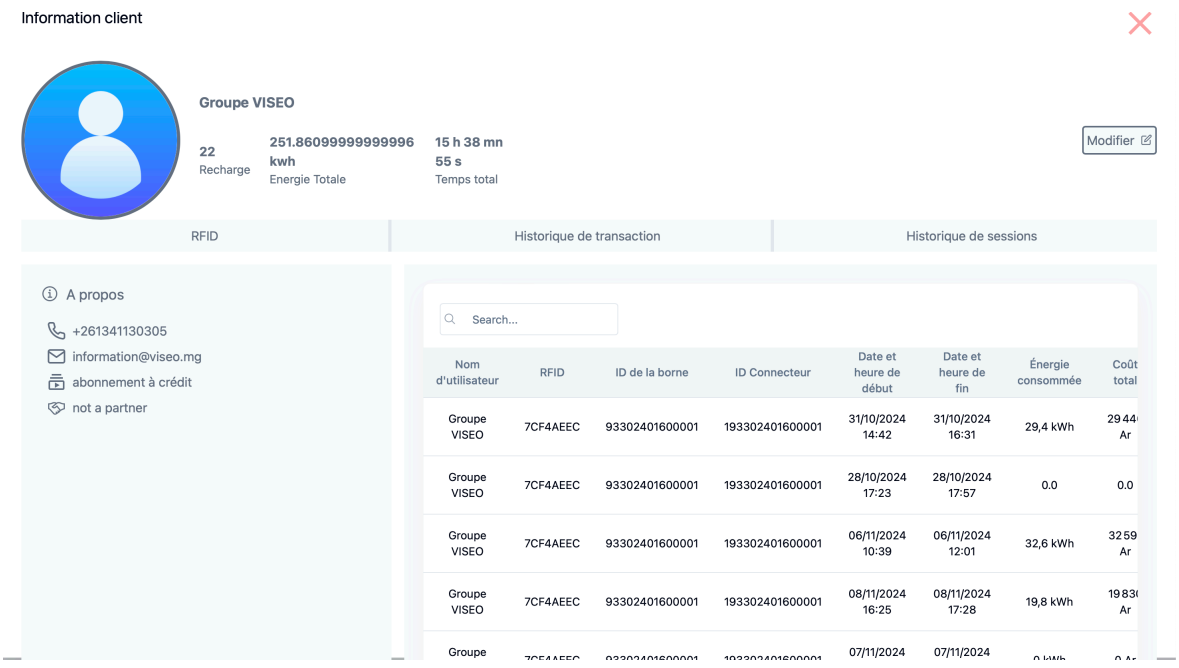


Figure 14 : Interface pour voir les détails du profile client

- **Modification des informations des utilisateurs**

© DAMA Prisca Fehiarisoa 2024

Les administrateurs peuvent mettre à jour les informations d'un utilisateur en utilisant un formulaire dédié, garantissant que les données restent pertinentes et à jour.

Voici l'interface de modification d'un utilisateur :

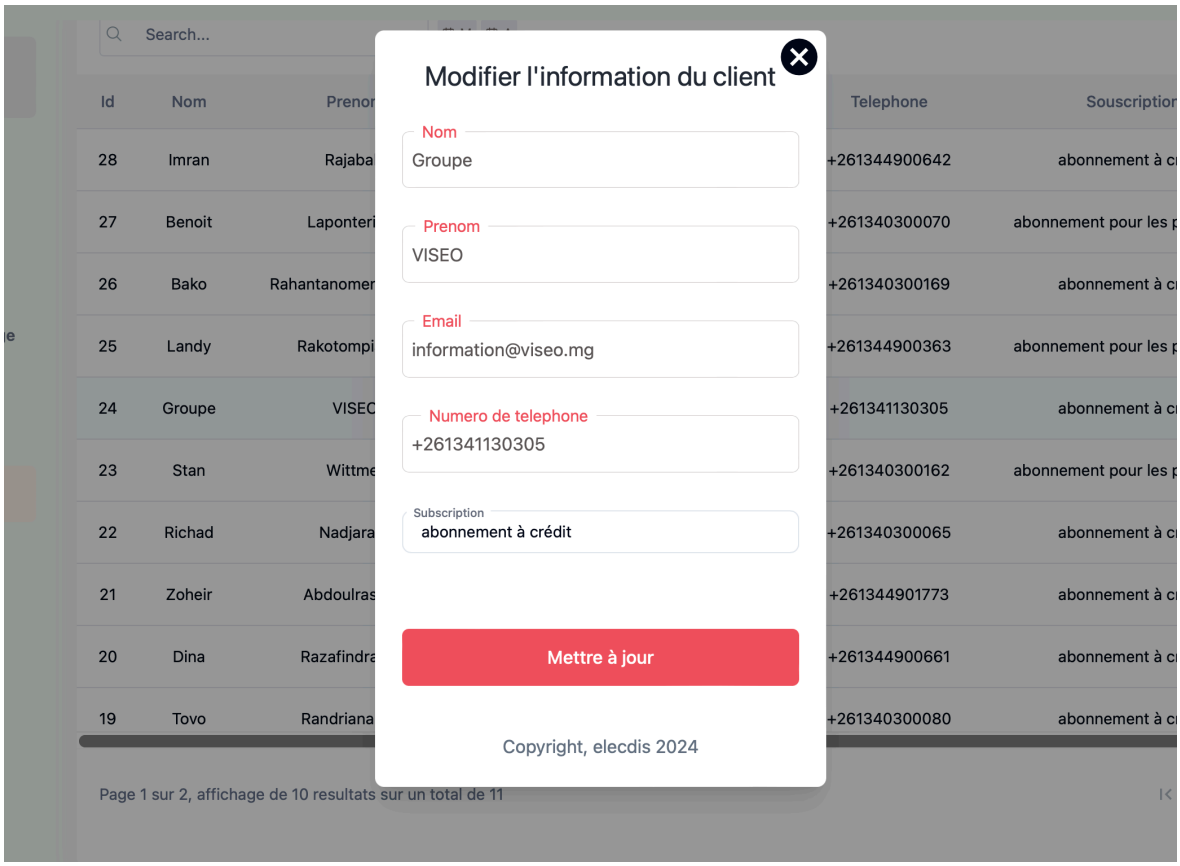


Figure 15 : interface de modification des Utilisateurs

- **Suppression d'un utilisateur :**

En cas de besoin, un administrateur peut supprimer un utilisateur du système. Cette fonctionnalité est utilisée lorsque l'utilisateur n'est plus actif ou qu'il y a une erreur dans l'enregistrement des informations.

© DAMA Prisca Fehiarisoa 2024

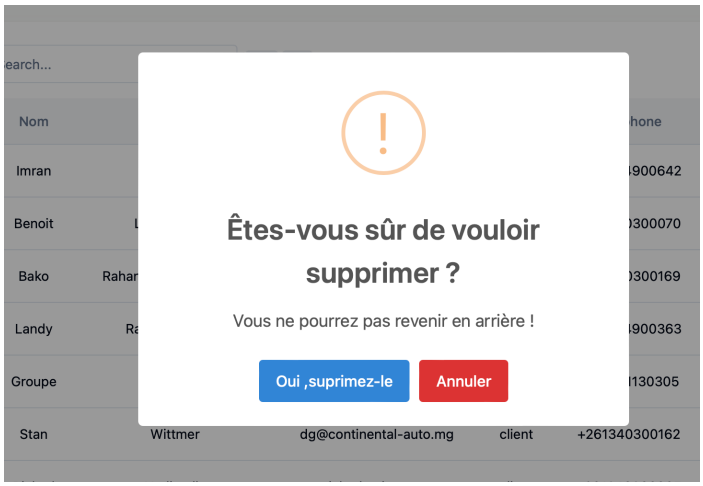


Figure 16 : Interface de confirmation de suppression d'un utilisateur

- **Création d'un compte utilisateur par le Super Administrateur**

Lors de la création d'un compte, chaque utilisateur se voit attribuer un ou plusieurs rôles en fonction des permissions qu'ils ont au sein de l'application. La création des comptes administrateurs est une tâche réservée aux « Super Administrateurs ».

2.2.2.2 Recherche d'Utilisateurs et Filtrage des Nouveaux Clients

Dans le cadre de la gestion des utilisateurs, deux fonctionnalités importantes ont été implémentées : la **recherche d'utilisateurs** et le **filtrage des nouveaux clients**. Ces outils permettent aux administrateurs de gérer efficacement les utilisateurs du système, en simplifiant leur identification et en facilitant le suivi des nouveaux utilisateurs.

- Recherche d'utilisateurs : Cette fonctionnalité permet aux administrateurs de trouver rapidement un utilisateur spécifique juste en utilisant des critères de recherches tels que leurs noms ou leurs e-mails.
- Filtrage des Nouveaux clients : Cette fonctionnalité permet d'isoler les nouveaux utilisateurs selon une période spécifique.

Voici l'interface de filtrage des clients par mois et par année :

© DAMA Prisca Fehiarisoa 2024

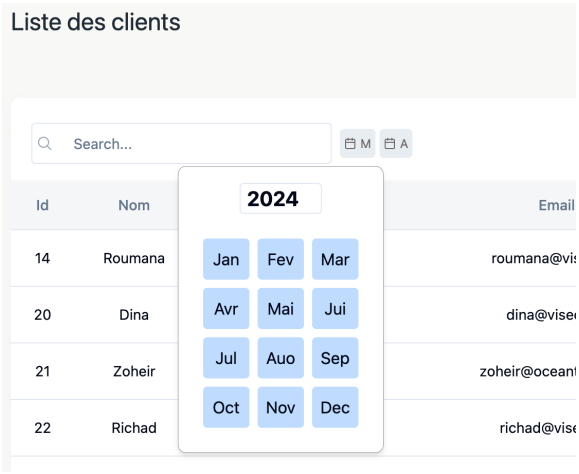


Figure 17 : Interface de filtrage des clients par mois et par années

2.2.2.3 Import des données des Utilisateurs en format CSV

Cette option permet aux administrateurs d'insérer une grande quantité d'informations sur les utilisateurs en une seule fois, en utilisant un fichier CVS contenant les informations des utilisateurs. Le formulaire suivant permet aux responsables d'effectuer l'importation des données :

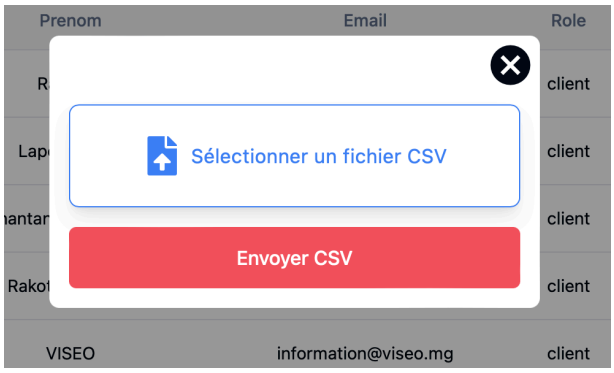


Figure 18 : Interface d'import CSV pour les utilisateurs

2.2.3 Gestion des RFID Utilisateurs

Pour mieux gérer l'utilisation des bornes de recharge, une solution a été proposée afin de donner plus d'autonomie aux utilisateurs et de faciliter le suivi des transactions pour les administrateurs. Cette solution repose sur l'utilisation de la carte RFID.

© DAMA Prisca Fehiarisoa 2024

Une carte RFID ou *Radio Frequency Identification* est un dispositif qui permet d'identifier un utilisateur de manière unique grâce à un identifiant spécifique. Ce dispositif agit comme une empreinte digitale qui signe l'identité de l'utilisateur.

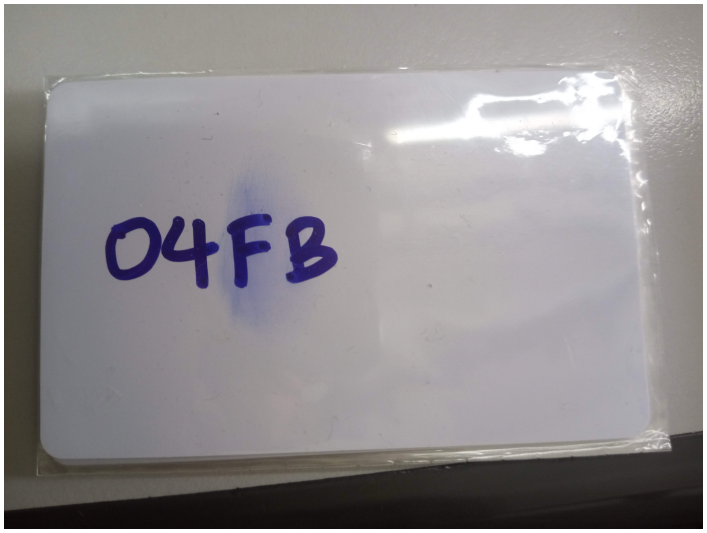


Figure 19 : Image d'une carte RFID utilisée par l'entreprise pour les recharges

Ce module englobe les fonctionnalités suivantes :

2.2.3.1 Enregistrement des cartes RFID

Cette fonctionnalité permet aux administrateurs d'enregistrer une carte RFID et l'associer à un utilisateur pour lui donner l'autorisation d'effectuer des recharges.



Figure 20 : Formulaire de création d'une carte RFID

© DAMA Prisca Fehiarisoa 2024

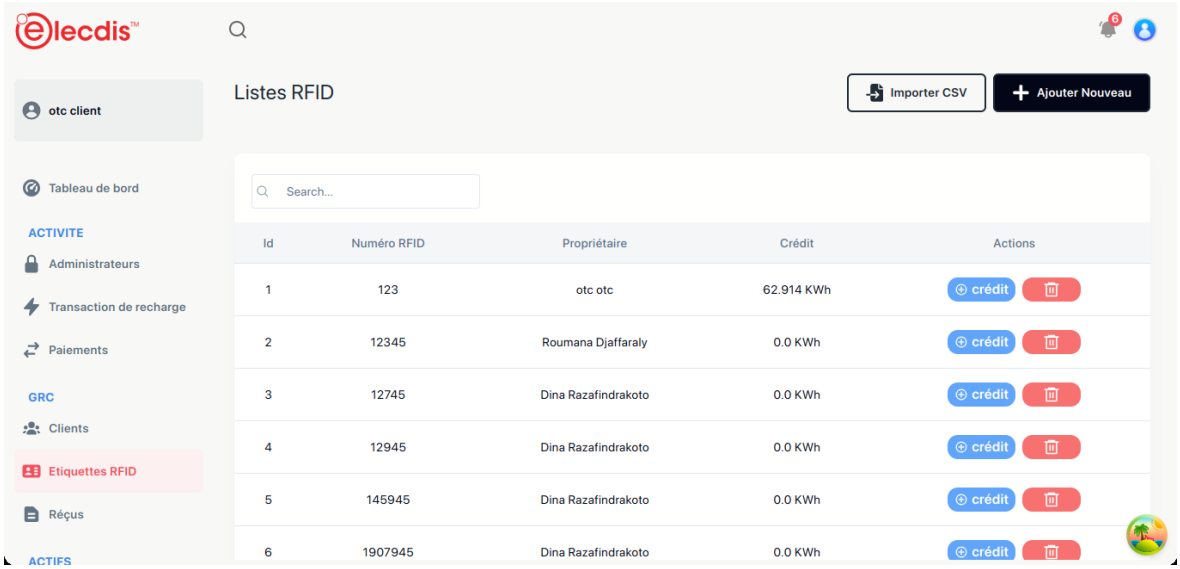


Figure 21 : Liste de toutes les cartes RFID

2.2.3.2 Import des données de la carte RFID sous forme de CSV

Ce procédé permet d'ajouter une masse de données de cartes RFID dans le système en important un fichier CSV. C'est un avantage important si on veut enregistrer des cartes RFID rapidement.



Figure 22 : Formulaire d'importation CSV des cartes RFID

2.2.3.3 Activation ou désactivation d'une carte

Cette option permet aux administrateurs d'activer une carte pour autoriser son utilisation ou la désactiver pour en interdire l'accès.

© DAMA Prisca Fehiarisoa 2024

2.2.3.4 Création d’un compte à crédit pour une carte RFID

Dans le système, chaque carte RFID peut être associée à un compte spécifique. Ce compte est alimenté en **kWh**(kilowattheures) et utilisé pour effectuer des sessions de recharge sur les bornes. Lorsqu'une carte RFID est utilisée pour une recharge, le crédit en kWh du compte associé à la carte est débité en fonction de la consommation d'énergie pendant la session.

Lors de la facturation, l'utilisation du compte à crédit dépend du type d'abonnement du client :

- Si le client possède un abonnement à crédit, le coût de la recharge (calculé en fonction du tarif appliqué et de la consommation en kWh) est directement déduit du compte à crédit de la carte RFID.
- Si le client n'a pas d'abonnement à crédit, il sera facturé en ariary (la devise locale) pour la consommation en fonction de la tarification horaire et du type d'énergie utilisé, et non par le biais du compte à crédit en kWh.

La création d'un compte à crédit pour une carte RFID permet d'assurer que chaque utilisateur peut approvisionner son compte avec un certain nombre de kWh, qu'il pourra utiliser ultérieurement pour recharger ses appareils via les bornes. Cette fonctionnalité garantit une gestion autonome et fluide des transactions liées à l’utilisation de la borne de recharge, où les crédits en kWh sont débités lors de chaque session, et l'utilisateur est averti de son solde restant.

Ce mécanisme simplifie la gestion des recharges, tout en offrant aux utilisateurs un moyen transparent de suivre leur consommation d'énergie et de maintenir un crédit suffisant pour leurs futurs besoins de recharge.

2.2.3.5 Scan RFID et vérification des autorisations lors des sessions de recharge

Lors du démarrage une session de recharge, l’utilisateur doit scanner sa carte RFID. La borne de recharge est équipée d’un module de scan RFID qui permet de lire les identifiants uniques de chaque carte lorsque l’utilisateur la présente. Cela déclenche le processus d’autorisation.

© DAMA Prisca Fehiarisoa 2024

Pour pouvoir fonctionner et communiquer avec le logiciel de gestion, les bornes de recharges envoient des messages via le protocole OCPP. Il existe plusieurs catégories de messages pour couvrir les principales opérations d'une borne de recharge. Dans phase d'autorisation, le message est appelé « Authorize ». Il est structuré comme sur la figure ci-dessous :

```
[2,"1000046","Authorize",{"idTag":"123"}]
```

Figure 23: Exemple d'un message OCPP : Authorize

Dans ce message :

- 2 indique le type du message
- "1000046" est un identifiant unique attribué au message
- "Authorize" représente la catégorie du message
- "idTag" est l'identifiant de la carte RFID scannée

Pour mieux visualiser le processus d'authentification par carte RFID, le diagramme de séquence ci-dessous illustre les interactions entre l'utilisateur, la borne de recharge et le centre de contrôle.

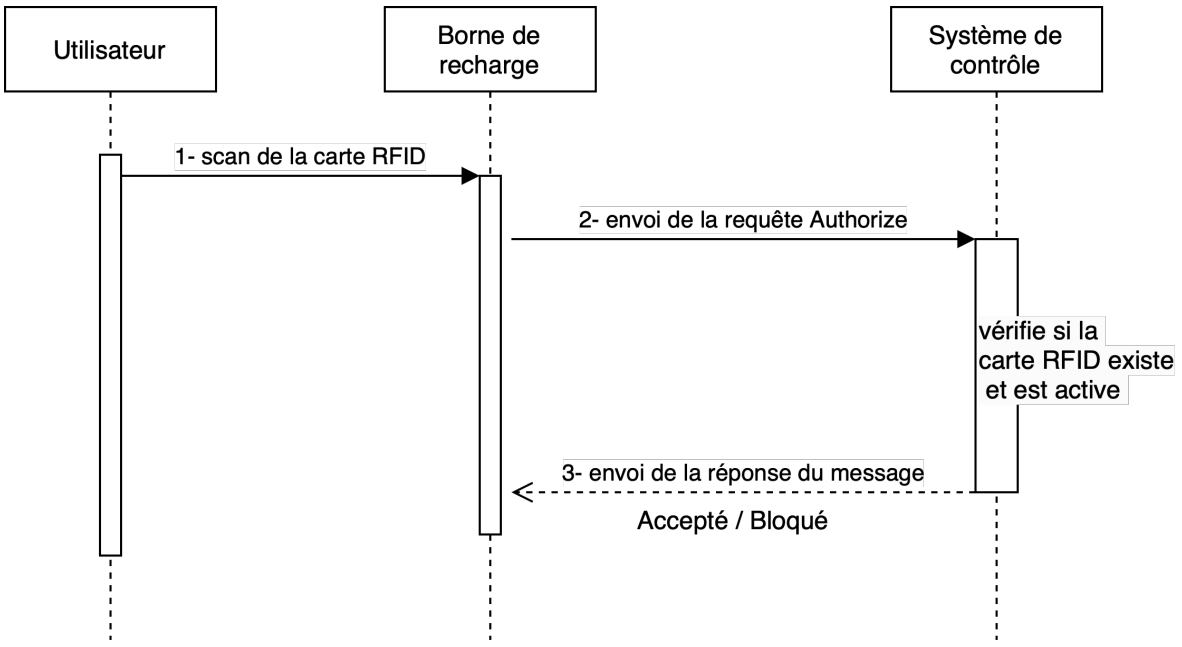


Figure 24 : Diagramme de Séquence - Processus d'Autorisation RFID

© DAMA Prisca Fehiarisoa 2024

Pour commencer, l'utilisateur scanne sa carte RFID sur le module de scan de la borne de recharge. Une fois la carte scannée, la borne envoie le message d'autorisation ci-dessous afin de vérifier l'identité de l'utilisateur et son droit d'accès :

[2,"1000046","Authorize",{"idTag":"123"}]

Cette vérification comprend plusieurs points importants, rentre autres :

- La vérification de l'existence de la carte RFID, et
- La vérification de l'activation de la carte

Une fois cette vérification terminée, le centre de contrôle peux envoyer deux types de réponses :

- Si la vérification est valide, l'utilisateur est autorisé à recharger et reçoit le message suivant :

[3,'1000046', {'idTagInfo' : {'status' :'Accepted','expiryDate' :'2024-11-01T09:07:19.227Z'}}]

- Par contre si une des vérifications a échoué, le message envoyé sera celle-ci :

[3, '1000046', {'idTagInfo' : {'status' : 'Blocked', 'expiryDate' : '2024-11-01T10:50:09.595Z'}}]

2.2.4 Gestion des sessions de recharges

Le module de gestion des sessions de recharges est important dans le cadre du projet, étant donné qu'il permet d'assurer un suivi précis des recharges initiés et effectuer ainsi une facturation transparente. Ce module comprend l'étape Enregistrement des sessions de recharge, la partie Gestion de la tarification dynamique, et la fonctionnalité Reprise d'une transaction en cas de coupure ou autres motifs.

2.2.4.1 Enregistrement des sessions de recharges

Parmi les différentes catégories de messages dans OCPP, trois d'entre eux sont impliqués dans une session de recharge : *StartTransaction, MeterValue* et *StopTransaction*.

Le diagramme suivant illustre le processus de recharge pour un véhicule électrique :

© DAMA Prisca Fehiarisoa 2024

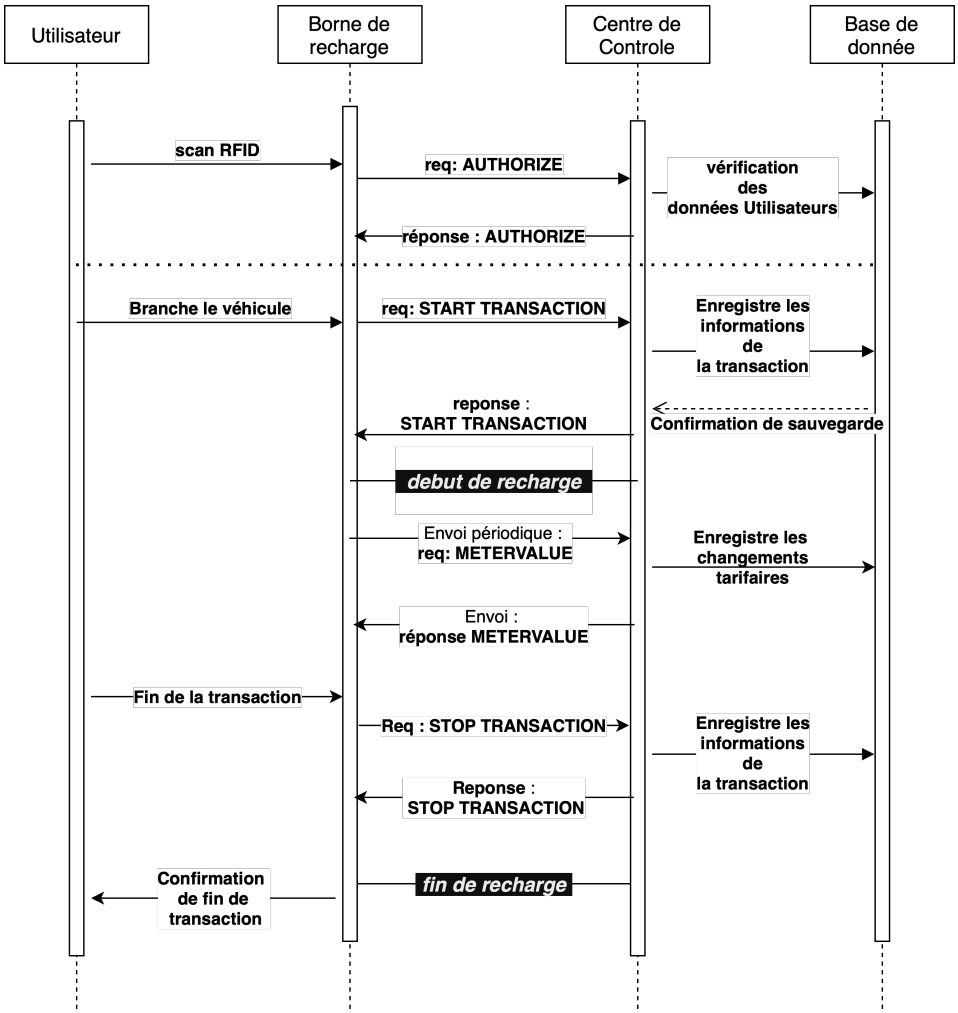


Figure 25 : Diagramme de séquence d'un processus de recharge

Avant l'initiation d'une nouvelle session de recharge, l'utilisateur doit d'abord scanner sa carte RFID afin de vérifier son identité et son accès vis-à-vis de la borne de recharge. Cela permet ainsi à l'utilisateur de brancher le véhicule après qu'il ait obtenu le droit de recharge. Lors de l'initialisation de la recharge, la borne envoie une requête « StartTransaction » puis une réponse correspondante lui est envoyée, ce qui démarre la recharge.

2.2.4.1.1 StartTransaction :

Ce message est envoyé par la borne de recharge au système de gestion lors du démarrage d'une session de recharge. Ce message contient les informations essentielles comme :

© DAMA Prisca Fehiarisoa 2024

- L’identifiant de la session : un identifiant unique attribué pour chaque session de recharge afin de référencer la transaction,
- L’identifiant du connecteur sur lequel la transaction a été initialisée,
- Le numéro RFID de la carte scanné au début de la transaction,
- La date de début de la transaction, et
- L’énergie initiale de la borne au début de la transaction.

L’envoi de ce message marque le début officiel de la session de recharge, ces données sont ensuite envoyées dans la base de données.

2.2.4.1.2 MeterValue

Une fois la transaction de recharge initiée, la borne de recharge envoie ce message à des intervalles réguliers afin de transmettre les données sur la consommation d’énergie durant la session. Cela inclut :

- L’identifiant de la transaction,
- La valeur en kWh de la consommation, et
- La date de la mesure.

Ce message est essentiel pour le calcul du coût de la session, car il permet d’obtenir des données précises sur la consommation d’énergie de la borne et les tarifs à appliquer. Chaque fois qu'un changement de tarif est détecté, celui-ci est enregistré dans la base de données afin de permettre le calcul des tarifs vers la fin de la transaction.

Le type d’abonnement de l’utilisateur peut influencer la méthode de tarification appliquée : abonnement à crédit ou tarif particulier.

2.2.4.1.3 StopTransaction

Ce dernier message marque l’arrêt de la transaction. Il contient les informations suivantes :

- Identifiant de la transaction,
- L’énergie finale : la mesure finale de l’énergie consommée durant la session,
- Le motif d’arrêt : indication du motif d’arrêt, et

© DAMA Prisca Fehiarisoa 2024

- La date de fin : le moment exact où se termine la transaction.

L'envoi de ce message conclut la session de recharge et déclenche le processus de facturation en utilisant les données collectées.

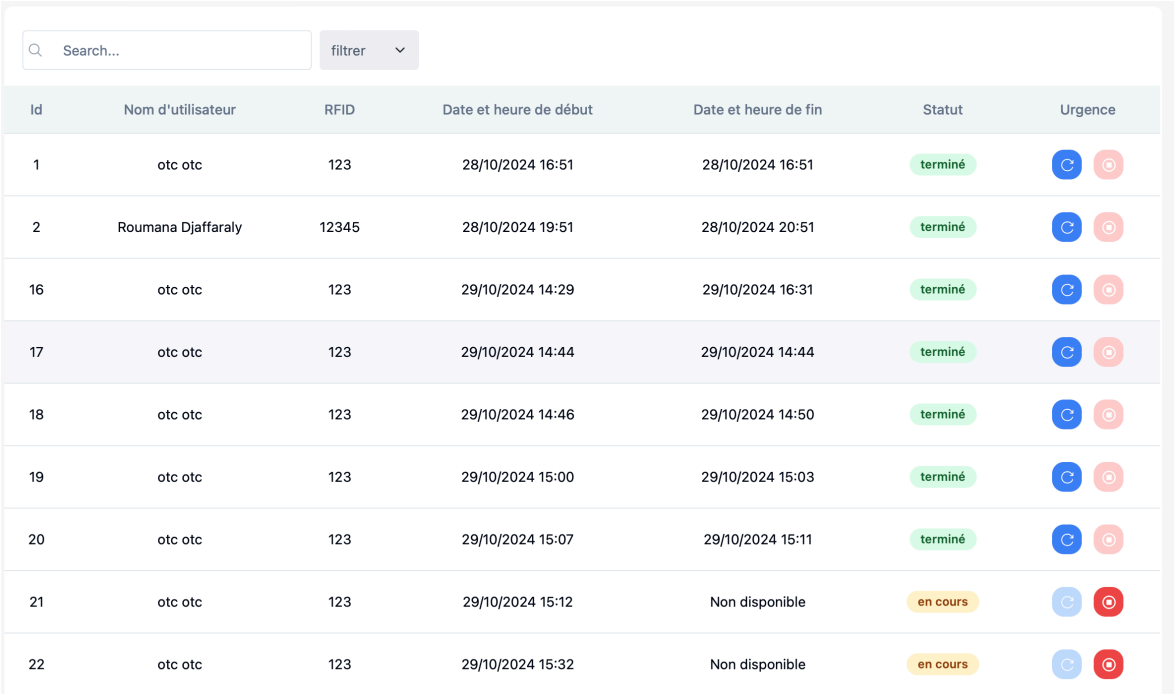


Figure 26 : Liste des transactions effectuées

2.2.4.2 Gestion de la tarification dynamique

Voici le mode de fonctionnement détaillé de la tarification pour toute recharge effectuée.

2.2.4.2.1 Comptes Clients

Un compte RFID peut être relié à un compte client contenant des crédits en kWh prépayés. Ce compte représente la quantité d'énergie dont le client dispose pour ses sessions de recharges.

2.2.4.2.2 Tarification par crédit

Pour les utilisateurs ayant un abonnement à crédit, le système applique un débit au compte relié à la carte RFID que l'utilisateur a utilisé durant la session. Ce débit est basé sur la consommation d'énergie multipliée par un facteur de majoration qui dépend de l'heure de recharge.

© DAMA Prisca Fehiarisoa 2024

Le tableau suivant représente les facteurs de majoration selon que ça soit une heure creuse, normale ou heure de pointe :

Période	Heures	Facteur de majoration
Heure creuse	0h - 7h, 22h - 24h	0.8
Heure normale	7h - 17h, 20h - 22h	1
Heure de pointe	17h - 20h	1.5

Tableau 5 : Tableau des facteurs de majoration

Supposons qu'un utilisateur avec un abonnement à crédit recharge son véhicule pendant une période de pointe, où le facteur de majoration est de 1,5. S'il consomme 10 kWh pendant cette session, voici le calcul du coût de la recharge.

- Consommation d'énergie : 10 kWh
- Facteur de majoration : 1,5
- Débit = Consommation d'énergie × Facteur de majoration = 10 kWh × 1,5 = 15 kWh

Ainsi, 15 kWh seront débités du compte RFID de l'utilisateur pour cette recharge en période de pointe. Le calcul se fera donc en fonction de ces critères, et la consommation en conséquence.

2.2.4.2.3 Tarification des particuliers

Pour les utilisateurs particuliers, la tarification est définie en fonction de plages horaires spécifiques selon les heures de pointes.

Imaginons maintenant qu'un particulier effectue une recharge de 8 kWh durant la période de pointe avec un tarif établi comme suit :

- Heure creuse (0h - 7h et 22h - 24h) : 3000 Ar/kWh
- Heure normale (7h - 17h et 20h - 22h) : 2000 Ar/kWh
- Heure de pointe (17h - 20h) : 4000 Ar/kWh

Le calcul du tarif en heure de pointe sera :

- Consommation (C) = 8 kWh
- Tarif (T) = 4000 Ar
- Coût Total = C * T = 8 * 4000 = 32000 Ar

© DAMA Prisca Fehiarisoa 2024

Ainsi l'utilisateur sera facturé à 32000 Ar pour une session de recharge effectuée pendant l'heure de pointe, une facture qui aurait été moindre si la session de recharge se faisait en période creuse.

2.2.4.3 Reprise d'une transaction en cas de coupure ou autres motifs

Étant donné les coupures de courant fréquentes à Madagascar, les sessions de recharges peuvent être interrompues de façon inattendue. Cette fonctionnalité de reprise d'une transaction en cas de coupure permet donc de sauvegarder l'état de la transaction afin de garantir que la recharge puisse être reprise manuellement dès que le courant est rétabli. Cela assure la continuité de la recharge et permet aux utilisateurs de reprendre la session au même point, en toute transparence. La figure suivante montre le bouton de reprise d'une transaction après une coupure de courant.



Figure 27 : Bouton de reprise d'une transaction

Parallèlement, la figure suivante représente les détails d'une session de recharge après la reprise de la transaction.

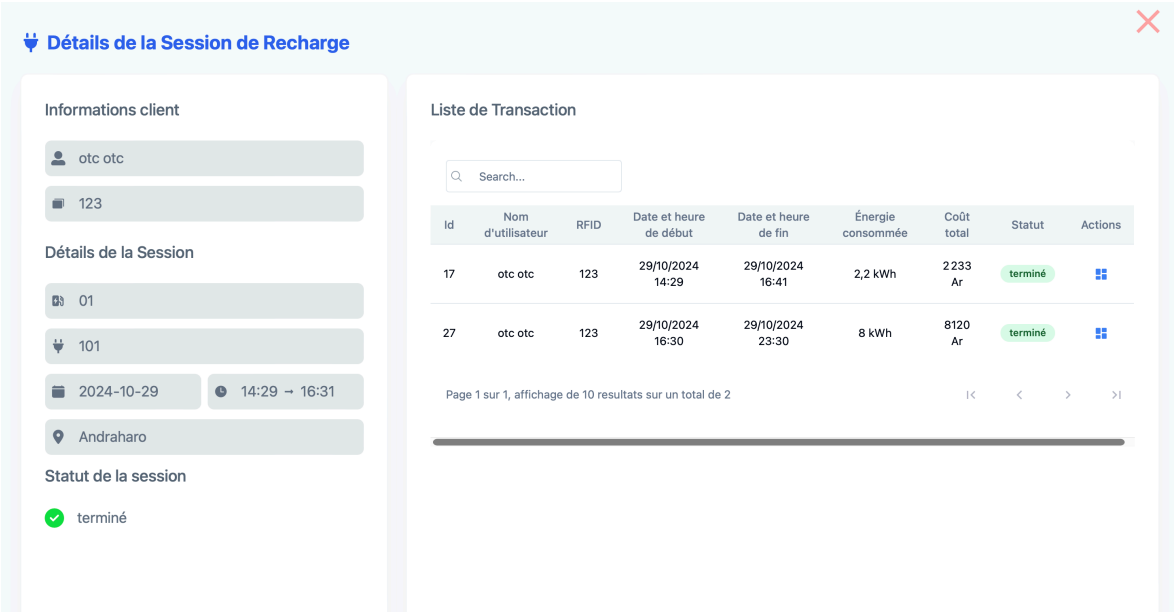


Figure 28 : Détails d'une session de recharge après reprise de la transaction

© DAMA Prisca Fehiarisoa 2024

2.3 État d'Analyse et Statistiques

La présente section fait état des différentes analyses, entre autres l'analyse de la fréquence d'utilisation des bornes, ainsi que celle des habitudes de recharge.

2.3.1 Analyse de la fréquence d'utilisation des bornes

Cette analyse permet aux administrateurs d'avoir une vue d'ensemble stratégique sur la fréquence d'utilisation des bornes de recharge.

Le graphe ci-après représente le nombre de client unique par heure et le nombre de sessions de recharge initiées. Il fournit des informations cruciales pour comprendre les habitudes de recharge tout au long de la journée. On peut ainsi identifier les heures de pointes où la demande de recharge est la plus élevée.

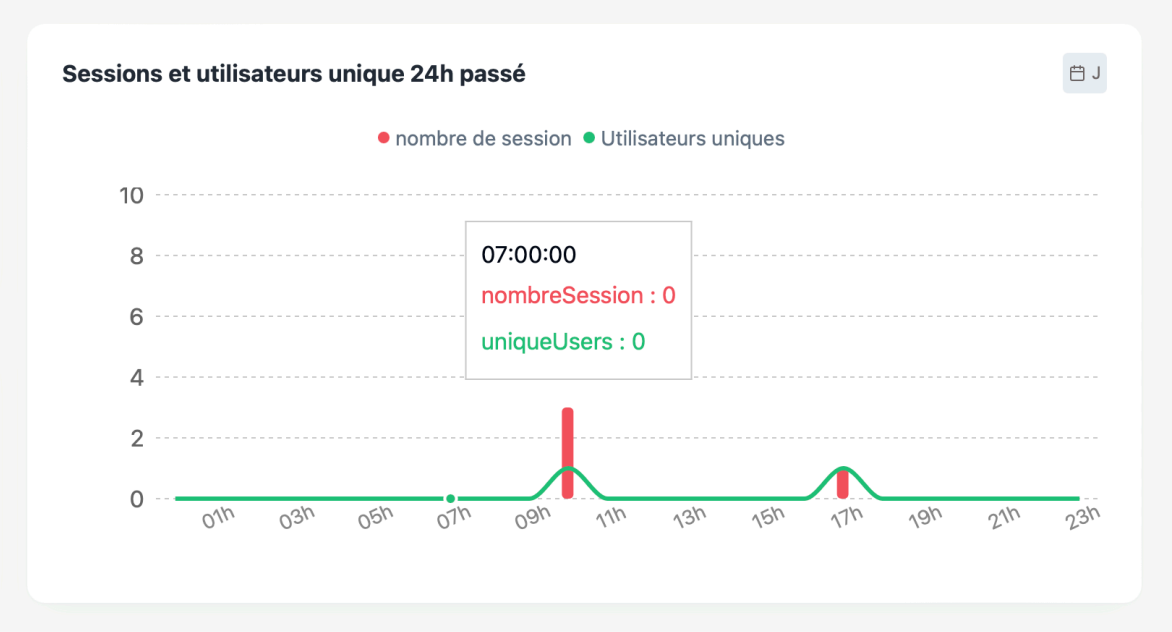


Figure 29 : Analyse de la fréquence d'utilisation des bornes

2.3.2 Analyse statistique des habitudes de recharge

Les statistiques sur l'heure de pointe et la durée moyenne des recharges offrent une vue détaillée des habitudes de consommation énergétique.

© DAMA Prisca Fehiarisoa 2024

L'identification de l'heure de pointe permet de savoir quand la demande de recharge est la plus élevée, ce qui est essentiel pour optimiser l'infrastructure et éviter la surcharge des bornes à ces moments critiques.

La durée moyenne des recharges révèle combien de temps les utilisateurs occupent une borne en moyenne.

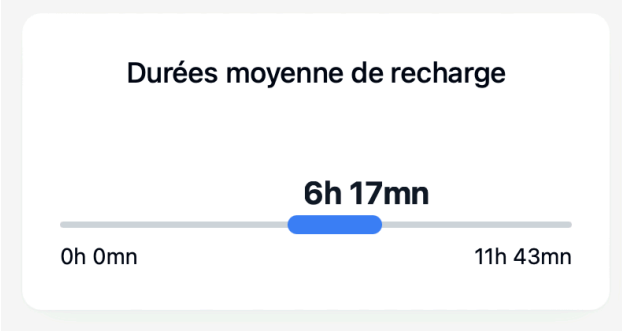


Figure 30 : Statistique de la durée moyenne de recharge



Figure 31 : Statistique de l'heure de pointe

2.4 Problèmes rencontrés et solutions

Au cours du développement du logiciel, des défis ont été rencontrés. Ces problèmes ont nécessité des solutions adaptées pour garantir le succès du projet. Cette section présente les principaux obstacles auxquels nous avons été confrontés, ainsi que les stratégies mises en œuvre pour les surmonter.

© DAMA Prisca Fehiarisoa 2024

2.4.1 Problème de coupures de courant lors d'une transaction

Lors des différents tests que nous avons réalisés, nous avons identifié un problème : la borne de recharge envoie immédiatement un message « StopTransaction » pour clore la session en cours, dès que le courant est rétabli, même si le véhicule reste connecté et que la charge n'est pas terminée. Cette situation pose un défi majeur, car elle entraîne une interruption inopinée de la recharge, laissant les utilisateurs avec des véhicules partiellement chargés, ce qui peut compromettre leur mobilité. De plus, pour relancer la transaction, il est impératif qu'une personne se déplace sur site, ce qui engendre des désagréments logistiques significatifs et nuit à l'expérience utilisateur. Ce dysfonctionnement souligne la nécessité d'une solution technique robuste pour garantir une gestion efficace et fiable des sessions de recharge.

Pour remédier à ce problème, nous avons implémenté une fonctionnalité de reprise de transaction de recharge, permettant aux administrateurs de relancer les sessions directement via l'interface de l'application. Cette solution est cruciale, car elle élimine la nécessité d'une intervention physique sur site, offrant ainsi une plus grande flexibilité et une efficacité dans la gestion des sessions de recharge. En permettant aux utilisateurs de continuer leur recharge sans interruption, nous améliorons considérablement l'expérience client tout en optimisant les opérations de l'entreprise.

2.4.2 Problème de configuration réseau dans Docker

Par défaut, les conteneurs Docker ne peuvent pas se voir directement s'ils ne sont pas sur le même réseau Docker. Pourtant les services que nous utilisions dépendaient les uns des autres. Pour remédier à ce problème, la création d'un réseau bridge personnalisé a été une solution, ce qui a permis de connecter les conteneurs à ce réseau afin qu'ils puissent communiquer entre eux.

© DAMA Prisca Fehiarisoa 2024

3 Évaluation du projet et connaissances acquises

Dans cette partie, nous présentons nos perspectives concernant le projet, à commencer par le bilan pour l'entreprise, mais également nos acquis personnels. Nous présentons par ailleurs vers la fin notre perspective d'amélioration de l'expérience client et ce que nous pensons être une alternative pour optimiser le système de gestion des recharges.

3.1 Bilan pour l'entreprise

L'intégration du module de gestion des bornes de recharge dans le système d'OceanTrade apporte un avantage stratégique majeur. Elle stimule les ventes des véhicules électriques en facilitant l'accès à un réseau de bornes de recharge, tout en permettant aux utilisateurs de suivre leurs sessions de recharge en temps réel. Cette initiative répond à un besoin crucial des conducteurs de véhicules électriques, tout en renforçant leur confiance et leur volonté d'adopter ces véhicules, grâce à une infrastructure fiable et accessible.

3.2 Bilan personnel

Ce stage au sein de l'OCEANTRADE m'a permis de mettre en pratique les connaissances que j'ai acquises au cours de mes études à l'IT University. J'ai eu l'opportunité d'acquérir de nouvelles compétences et connaissances dans plusieurs domaines clés, l'une des nouvelles compétences que j'ai acquises étant la maitrise du protocole OCPP grâce à son intégration dans le cadre du projet.

Cette expérience m'a aussi permis de découvrir le monde du travail en entreprise, d'appréhender les enjeux réels des projets en entreprise. Cette immersion m'a surtout offert une vision plus claire du fonctionnement des projets dans un environnement professionnel et m'a préparé à des responsabilités futures.

3.3 Extension et évolution de l'application

Pour améliorer l'expérience client et optimiser le système de gestion de recharge, plusieurs évolutions sont à envisager, je propose les alternatives suivantes :

- La création d'une application Mobile destinée aux clients qui leur permettraient de suivre leurs sessions de recharge, leurs consommations d'énergies et les dépenses qu'ils ont effectuées, offrant ainsi une transparence complète aux clients ;

© DAMA Prisca Fehiarisoa 2024

- L’enregistrement des RFID via NFC (Near Field Communication), une technologie de communication sans fil à courte portée qui permet à deux appareils compatibles de communiquer lorsqu'ils sont rapprochés, généralement à moins de 10 cm. L’intégration de ce module dans l’application mobile permettrait aux utilisateurs et aux administrateurs d’enregistrer facilement leur carte RFID via leur téléphone, simplifiant ainsi la gestion des cartes RFID.

© DAMA Prisca Fehiarisoa 2024

Conclusion

Ce projet de développement d'un logiciel de gestion des bornes de recharge pour véhicule électrique constitue une avancée majeure pour OceanTrade dans sa quête d'une infrastructure de mobilité durable. En intégrant des fonctionnalités tels que la gestion des utilisateurs, l'utilisation des cartes RFID, le suivi des sessions de recharges et un module de tarification, l'application optimise la gestion des bornes.

Actuellement en phase de préproduction, l'application est déjà testée par les employés de l'entreprise, permettant ainsi d'ajuster ses fonctionnalités avant son déploiement complet. Cette expérience a été particulièrement enrichissante pour moi, car elle m'a permis d'appliquer mes connaissances tout en développant des compétences techniques précieuses.

Les perspectives d'avenir sont prometteuses, avec des idées telles que le développement d'une application mobile pour les clients et l'intégration de technologies innovantes comme la NFC. Grâce à ces efforts, OceanTrade est bien positionnée pour soutenir l'adoption des véhicules électriques à Madagascar, rendant cette technologie plus accessible et fiable.

En somme, ce projet ne représente pas seulement une avancée technologique, mais aussi un pas vers un avenir plus durable pour Madagascar.

© DAMA Prisca Fehiarisoa 2024

Bibliographie

1 - Openchargealliance.org | Présentation du protocole OCPP.

Disponible sur : https://openchargealliance.org/protocols/open-charge-point-protocol/ (consultée le 23-09-2024)

2 - Wikipedia.org | Open charge Point Protocol.

Disponible sur : https://en.wikipedia.org/wiki/Open_Charge_Point_Protocol (consultée le 23-09-2024)

3 - Youtube.com | RFID, qu'est-ce que c'est :

Disponible sur : https://www.youtube.com/watch?V=tubtpmnqhcc (consultée le 29-10-2024)

4 - Medium.com | OCPP, le guide complet :

Disponible sur : https://medium.com/@yocharge2022/open-charge-point-protocol-ocpp-a-complete-guide-cfcfb57c4108 (consultée le 14-10-2024)

5 - Cloudamqp.com | rabbitmq Exchanges, routing keys and bindings

Disponible sur : https://www.cloudamqp.com/blog/part4-rabbitmq-for-beginners-exchanges-routing-keys-bindings.html (consulté le 11-10-2024)

6 - Docker.com | Getting started with docker:

Disponible sur : https://www.docker.com/blog/getting-started-with-docker-desktop/ (consulté le 22-08-2024)

7 - Medium.com | Getting started : websocket

Disponible sur : https://medium.com/@nmjoshi/getting-started-websocket-with-fastapi-b41d244a2799 (Consultée le 23-08-2024)

8 - Fastapi.tiangolo.com | Tutoriel- Guide utilisateur – fastapi

Disponible sur : https://fastapi.tiangolo.com/fr/tutorial/#installer-fastapi (consultée le 23-08-2024)

© DAMA Prisca Fehiarisoa 2024

9 - Geeksforgeeks.org | Authentification and authorization with fastapi

Disponible sur : https://www.geeksforgeeks.org/authentication-and-authorization-with-fastapi/ (consultée le 27-08-2024)

10 - Fastapi.tiangolo.com | fastapi - Security – First Step

Disponible sur : https://fastapi.tiangolo.com/tutorial/security/first-steps/#check-it (consultée le 27-08-2024)

11 - Softwareag.com | python vs Java:

Disponible sur : https://www.softwareag.com/en_corporate/blog/streamsets/python-vs-java-comparison.html (consultée le 26-08-2024)

12 - Snaplogic.com | python vs java quelle est la différence ?

Disponible sur : https://www.snaplogic.com/fr/glossary/python-vs-java (consultée le 26-08-2024)

13 - Itta.net | Angular vs React

Disponible sur : https://www.itta.net/blog/angular-vs-react-js-quel-est-le-meilleur-framework-en-2024/ (consultée le 26-08-2024)

14 - Dipolerfid.fr | *Les étiquettes rfid et dans quels domaines s'appliquent-elles*

Disponible sur : https://www.dipolerfid.fr/blog-rfid/etiquettes-rfid-et-dans-quels-domaines-appliquent-elles (consultée le 2-10-2024)

© DAMA Prisca Fehiarisoa 2024